

Transformasi Data menjadi Representasi yang Bermakna

Course:

Matrikulasi S3 Ilmu Komputer FILKOM UB

by:

Yuita Arum Sari, S.Kom., M.Kom., Ph.D.

yuita@ub.ac.id

Versi Dokumen:

Tanggal: 15; Bulan: September; Tahun: 2026

© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id

Mengapa “representation” adalah inti AI?

Algoritma tidak melihat gambar, dokumen, atau suara. Ia hanya melihat representasi numerik.

OBJEK YANG KITA PAHAMI

YANG DILIHAT MODEL



Citra wajah



vektor piksel

$28 \times 28 = 784$ nilai intensitas



Dokumen ilmiah



vektor frekuensi/token

kosakata ≈ 30.000 dimensi



Rekaman suara



spektrogram/*feature vector*

128 band $\times$ 300 frame



Aktivitas sensor



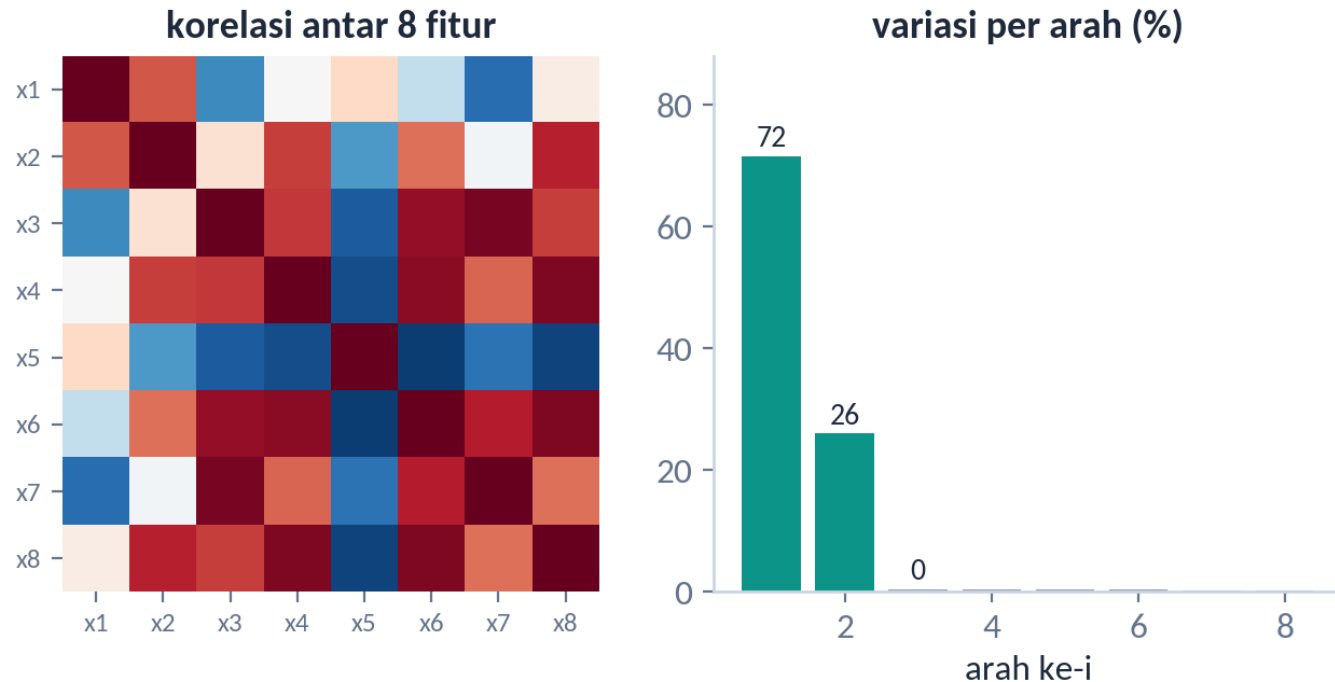
time-series matrix

6 sensor $\times$ 1000 langkah

Kualitas representasi sering menentukan seberapa mudah pola dapat dipelajari.

Masalah utama: dimensi tinggi $\neq$ informasi tinggi

Banyak fitur dapat redundan, berkorelasi, *noisy*, atau tidak relevan.



Redundan

Beberapa kolom membawa informasi yang sama.

Berkorelasi

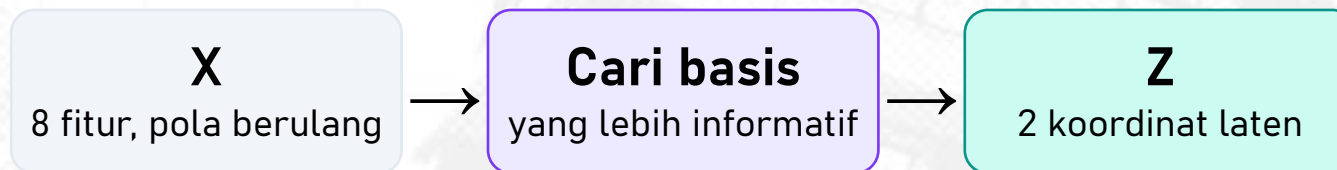
Fitur bergerak bersama sehingga koefisien tidak stabil.

Noisy

Sebagian variasi hanya gangguan pengukuran.

Tidak relevan

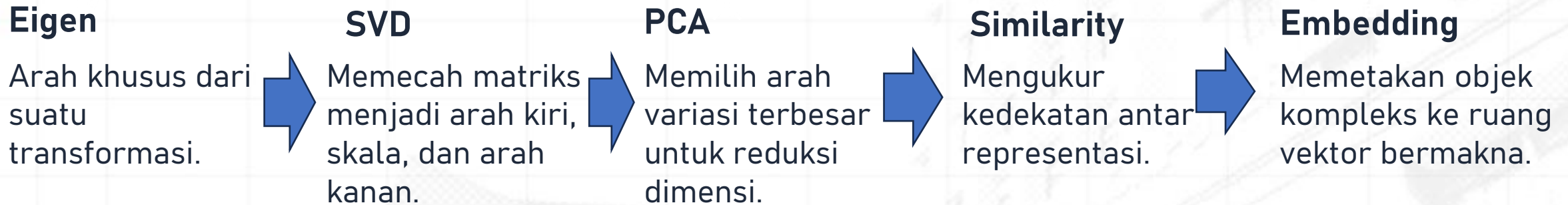
Tidak membantu tugas yang dituju.



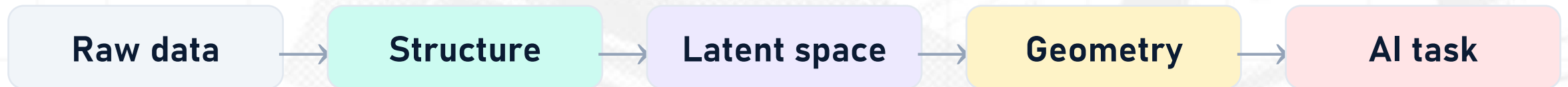
Dimensionality reduction bukan sekadar “membuang kolom”, tetapi mencari koordinat baru yang mempertahankan struktur penting.

Satu benang merah untuk seluruh pertemuan

konsep terlihat berbeda, tetapi semuanya berbicara tentang geometri data.



ALUR BESARNYA



Benang merahnya adalah perubahan basis dan geometri ruang representasi.

Prasyarat dan notasi

Ringkasan aljabar linear yang dipakai sepanjang sesi. Simpan sebagai rujukan cepat.

$$\mathbf{x} \in \mathbb{R}^n$$

vektor

daftar n bilangan;
titik/arrah di ruang n-
dimensi

$$\mathbf{x}^T \mathbf{y}$$

dot product

$$\sum x_i y_i = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$$

$$\|\mathbf{x}\|_2$$

norm Euclidean

$\sqrt{\mathbf{x}^T \mathbf{x}}$: panjang
vektor

$$\mathbf{A} \in \mathbb{R}^{m \times n}$$

matriks

transformasi linear
dari $\mathbb{R}^n$ ke $\mathbb{R}^m$

$$\mathbf{x}^T \mathbf{y} = 0$$

ortogonal

dua arah saling tegak
lurus

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$$

basis ortonormal

kolom saling
ortogonal, panjang 1:
rotasi/refleksi

$$\text{rank}(\mathbf{A})$$

rank

banyaknya kolom
bebas linear

$$\mathbf{A} = \mathbf{A}^T$$

matriks simetris

kunci: eigenvalue
real, eigenvector
ortogonal

01

PART I · EIGEN

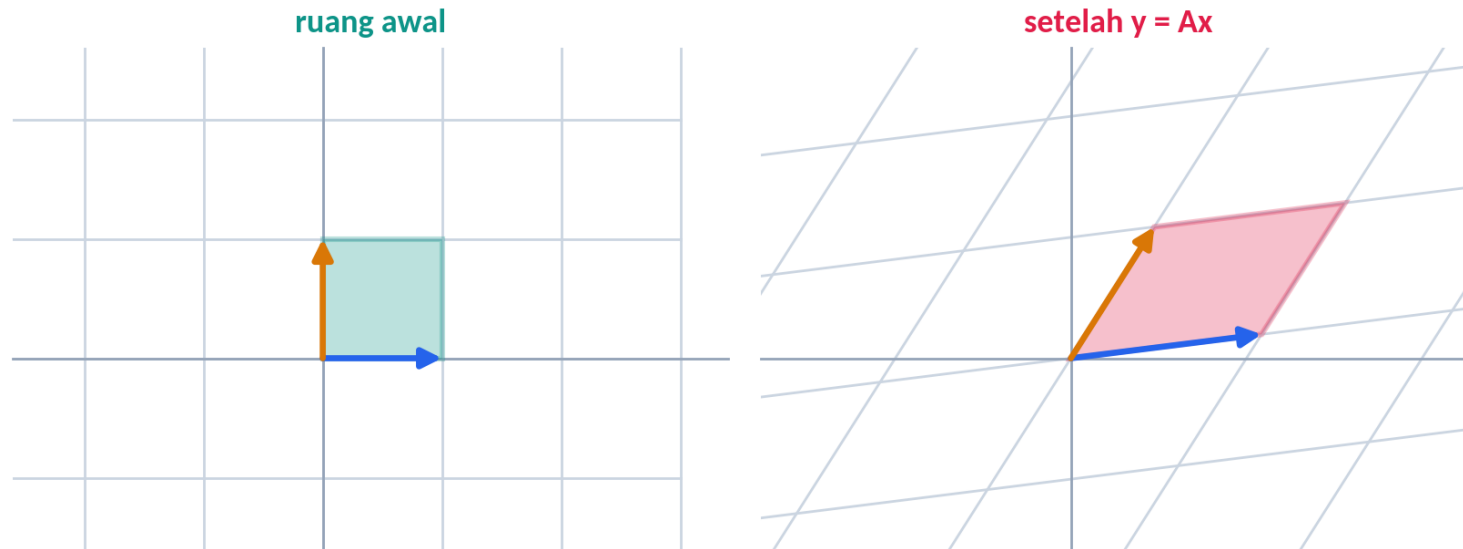
Eigen: arah yang bertahan

$$Av = \lambda v$$

- 1 Transformasi linear mengubah ruang
- 2 Definisi eigenvector dan eigenvalue
- 3 Persamaan karakteristik
- 4 Eigenvector sebagai basis baru
- 5 Diagonalisasi dan power iteration
- 6 Matriks simetris dan bentuk kuadrat

Transformasi linear mengubah ruang

Matriks A dapat meregangkan, memutar, mencerminkan, atau menggeser arah melalui kombinasi linear.



$$y = Ax$$

Peregangan

panjang berubah per arah

Rotasi/refleksi

arah berputar atau terbalik

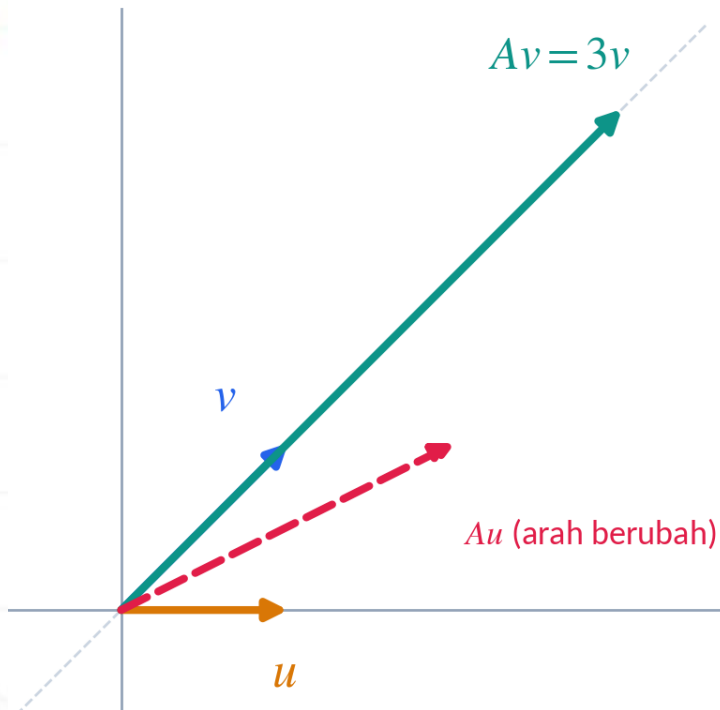
Shear

kotak menjadi jajar genjang

Eigenvector adalah arah istimewa yang tidak berubah arah oleh A.
Hanya panjang atau tandanya yang berubah.

Definisi eigenvector dan eigenvalue

Untuk matriks persegi A , vektor $v \neq 0$ disebut eigenvector jika hasil transformasinya hanya berupa penskalaan.



$$Av = \lambda v$$

v
arah khusus

λ
faktor skala

A
transformasi

$\lambda > 1$



v

$0 < \lambda < 1$



v

$\lambda < 0$



v

$\lambda = 0$



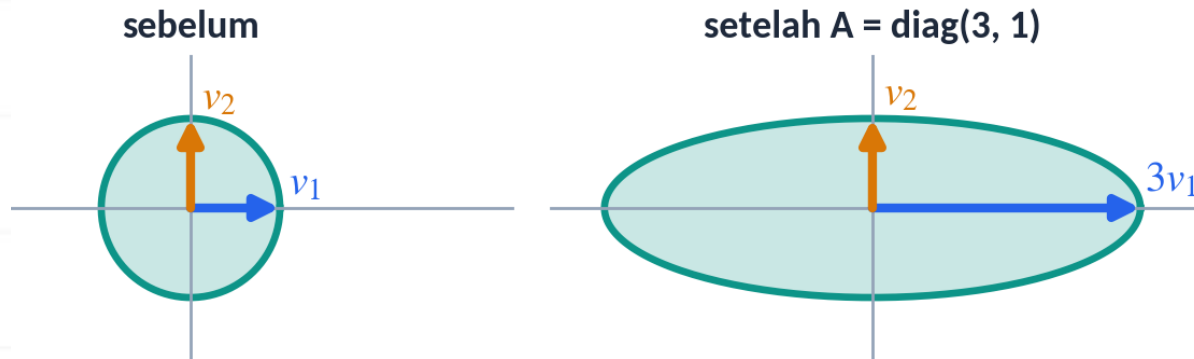
v

$\lambda > 1$ diregangkan · $0 < \lambda < 1$ diperkecil · $\lambda < 0$ berbalik · $\lambda = 0$ dipetakan ke nol

Eigenvector hanya ditentukan sampai skala: jika v eigenvector, maka cv juga ($c \neq 0$). Karena itu biasanya dinormalisasi.

Contoh eigen yang bisa dihitung mental

Gunakan transformasi diagonal agar makna geometrinya terlihat.



$$A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \rightarrow A v_1 = 3 v_1$$

$$v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \rightarrow A v_2 = 1 v_2$$

A meregangkan sumbu-x 3x, tetapi mempertahankan sumbu-y.

Pada matriks diagonal, basis standar langsung menjadi eigenvector. Pada matriks umum, kita harus mencari arah khusus itu.

Dari $Av = \lambda v$ ke persamaan karakteristik

Pindahkan semua ke satu sisi, lalu tanyakan kapan sistem punya solusi tak-nol.

Mulai

1 $Av = \lambda v \Rightarrow (A - \lambda I)v = 0$

Syarat solusi $v \neq 0$

2 $\det(A - \lambda I) = 0$

Hasil

3 polinom derajat n dalam $\lambda \rightarrow$ eigenvalue

Untuk tiap λ

4 selesaikan $(A - \lambda I)v = 0 \rightarrow$ eigenvector

Mengapa singular?

Agar ada solusi non-trivial $v \neq 0$, matriks $(A - \lambda I)$ harus singular. Eigenvalue muncul ketika transformasi kehilangan invertibilitas setelah dikurangi λI .

Cek cepat

$$\text{tr}(A) = \sum \lambda_i \text{ dan } \det(A) = \prod \lambda_i$$

Matriks $n \times n$ memiliki n eigenvalue (dengan multiplisitas), yang bisa kompleks.

Contoh lengkap 2×2

Cari eigenvalue dan eigenvector dari $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

$$\det([2-\lambda, 1; 1, 2-\lambda]) = 0$$

$$(2-\lambda)^2 - 1 = 0$$

$$\lambda^2 - 4\lambda + 3 = 0$$

$$\lambda_1 = 3, \quad \lambda_2 = 1$$

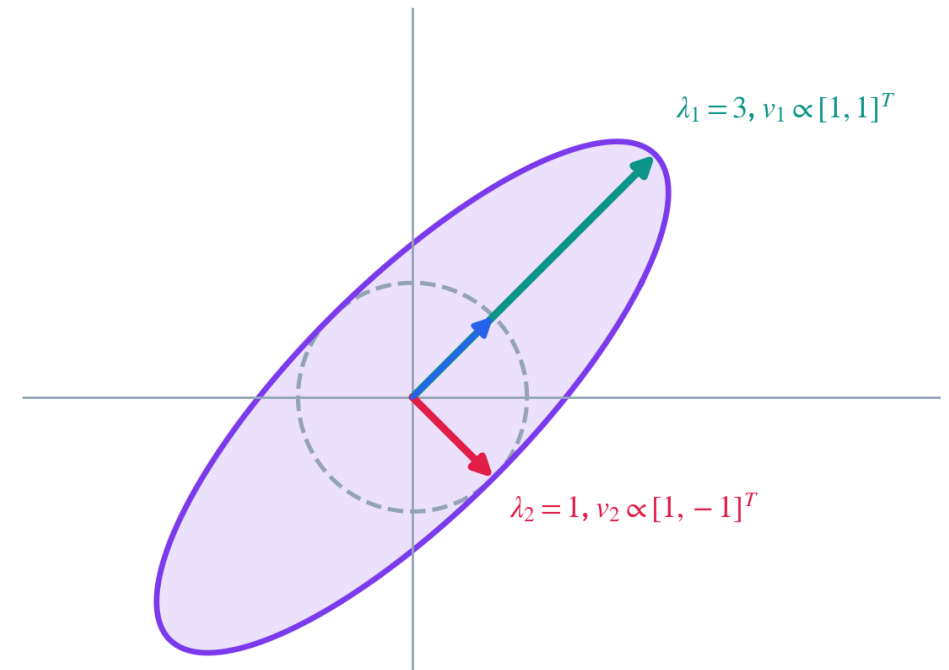
$$\lambda = 3 \rightarrow v \propto [1, 1]^T \text{ (diagonal naik)}$$

$$\lambda = 1 \rightarrow v \propto [1, -1]^T \text{ (diagonal turun)}$$

Verifikasi

$$\text{tr}(A) = 4 = 3 + 1 \quad \text{dan} \quad \det(A) = 3 = 3 \cdot 1$$

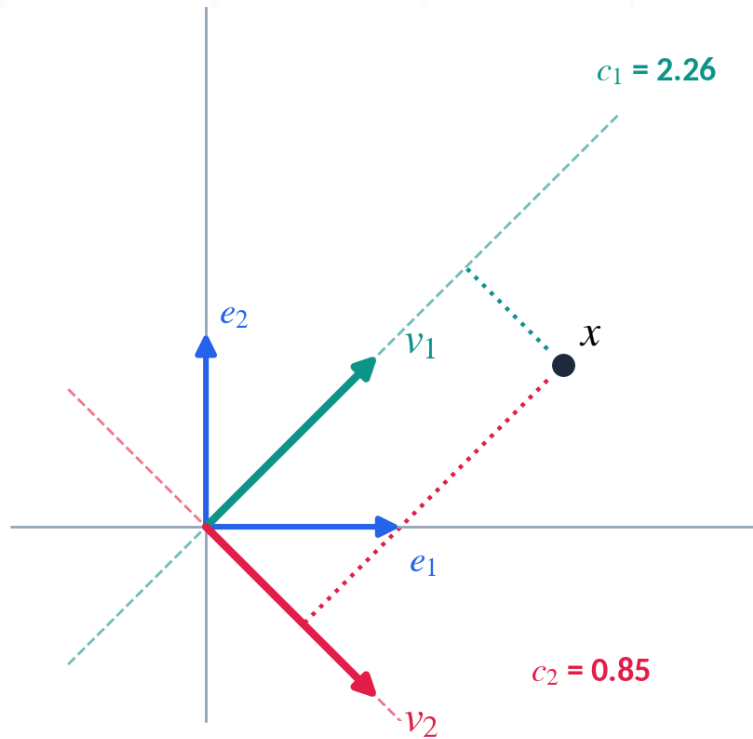
Dua eigenvector membentuk dua arah utama transformasi: diagonal naik dan diagonal turun.



Lingkaran satuan (putus-putus) → elips: sumbu utama searah eigenvector.

Eigenvectors sebagai basis baru

Alih-alih memakai sumbu x_1, x_2 , kita dapat mendeskripsikan data dengan arah eigenvector.



$$x = c_1 v_1 + c_2 v_2$$

$$c = Q^T x \quad (Q = [v_1 \ v_2] \text{ ortonormal})$$

	koordinat lama	koordinat baru
x	(2.20, 1.00)	(2.26, 0.85)
arti	x_1, x_2	z_1, z_2



Insight

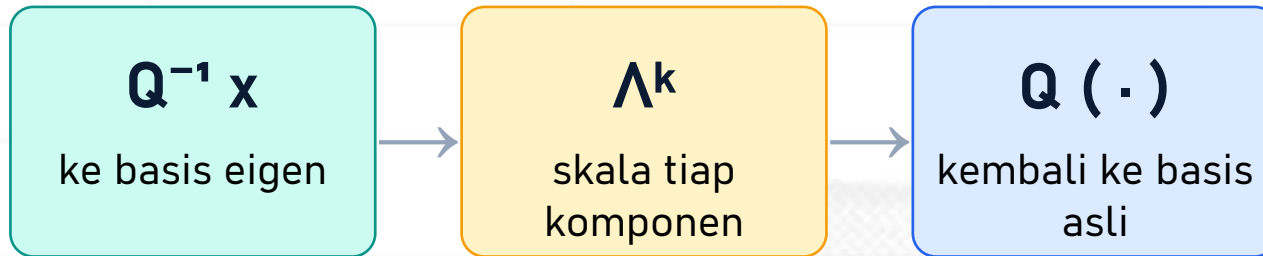
Representasi laten sering berarti: objek yang sama, tetapi diekspresikan dalam koordinat atau basis yang lebih berguna.

Objek x sama; hanya “penggaris”-nya yang berganti.

Diagonalisasi dan *power iteration*

Di basis eigen, transformasi hanya menskalakan tiap koordinat. Itulah mengapa pangkat matriks menjadi mudah.

$$A = Q \Lambda Q^{-1} \Rightarrow A^k = Q \Lambda^k Q^{-1}$$



Komponen dominan menang

Dengan $\lambda_1 = 3$ dan $\lambda_2 = 1$, komponen v_1 tumbuh 3^k sedangkan v_2 tetap, sehingga arah $A^k x$ cepat mendekati v_1 .

k	$A^k x_0$ (dinormalisasi)	cos dengan v_1
0	[1.000, 0.000]	0.7071
1	[0.894, 0.447]	0.9487
2	[0.781, 0.625]	0.9939
3	[0.733, 0.680]	0.9993
4	[0.716, 0.698]	0.9999
5	[0.710, 0.704]	1.0000

Aplikasi

Power iteration: PageRank, eigenvector skala besar. Nanti akan muncul lagi saat menghitung PCA pada data besar.

$$x_0 = [1, 0]; A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Kapan eigen decomposition tersedia?

Tidak semua matriks berperilaku seindah contoh simetris.

Kasus yang nyaman

- A simetris real $\rightarrow$ eigenvalue real
- Eigenvector dapat dipilih ortonormal
- $A = Q \Lambda Q^T$
- Penting untuk covariance matrix

Kasus umum

- Eigenvalue dapat kompleks
- Tidak selalu punya cukup eigenvector independen
- Dekomposisi eigen hanya untuk matriks persegi
- Untuk matriks data $m \times n$, SVD jauh lebih universal

Inilah alasan SVD menjadi alat fundamental untuk data science: ia berlaku pada setiap matriks real $m \times n$.

Matriks simetris dan bentuk kuadrat

Teorema spektral: bentuk yang paling penting untuk data karena covariance selalu simetris.

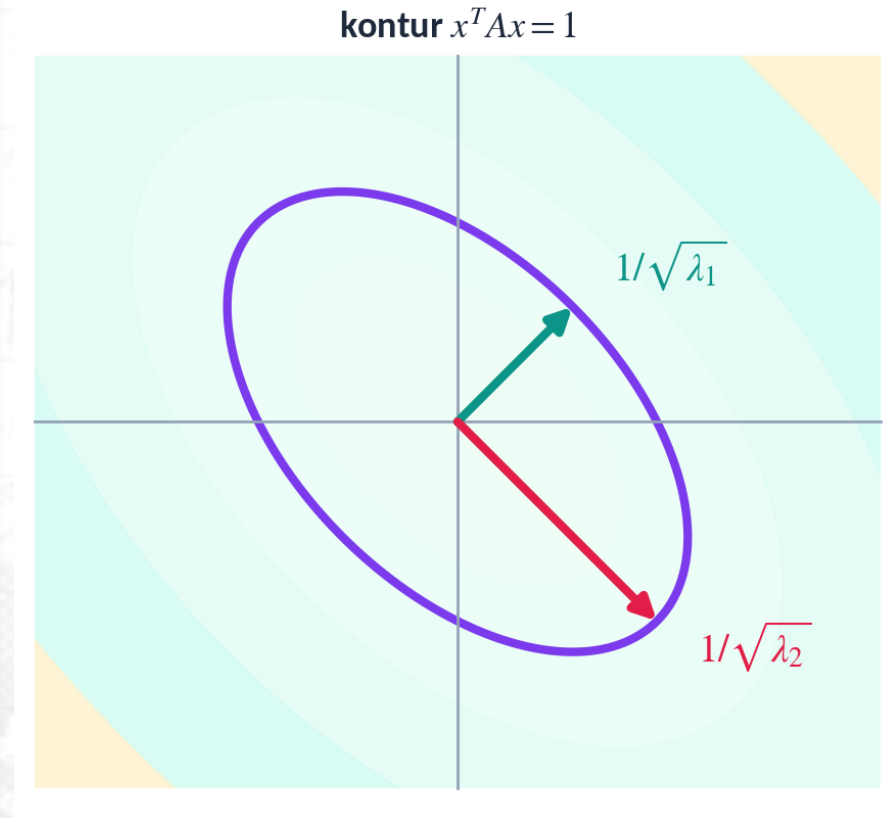
$$A = A^T \Rightarrow A = Q \Lambda Q^T$$

$$x^T A x = \sum_i \lambda_i c_i^2, \quad c = Q^T x$$

Jenis	Syarat eigenvalue	Contoh
Positive definite	semua $\lambda > 0$	covariance penuh rank
Positive semidefinite	semua $\lambda \geq 0$	covariance umum, $A^T A$
Indefinite	tanda campur	Hessian di saddle

Kunci PCA

Covariance dan $A^T A$ selalu positive semidefinite, sehingga eigenvalue real dan ≥ 0 serta eigenvector ortonormal.



02

PART II · SVD

SVD: membedah matriks

$$A = U \Sigma V^T$$

- 1 Rotate → scale → rotate
- 2 Dimensi U, Σ , V dan hubungan dengan eigen
- 3 Contoh numerik 2x2
- 4 Penjumlahan komponen rank-1
- 5 Low-rank approximation dan Eckart-Young
- 6 Contoh citra, energi, dan pemilihan k
- 7 Rank, intrinsic dimension, pseudo-inverse

Singular Value Decomposition

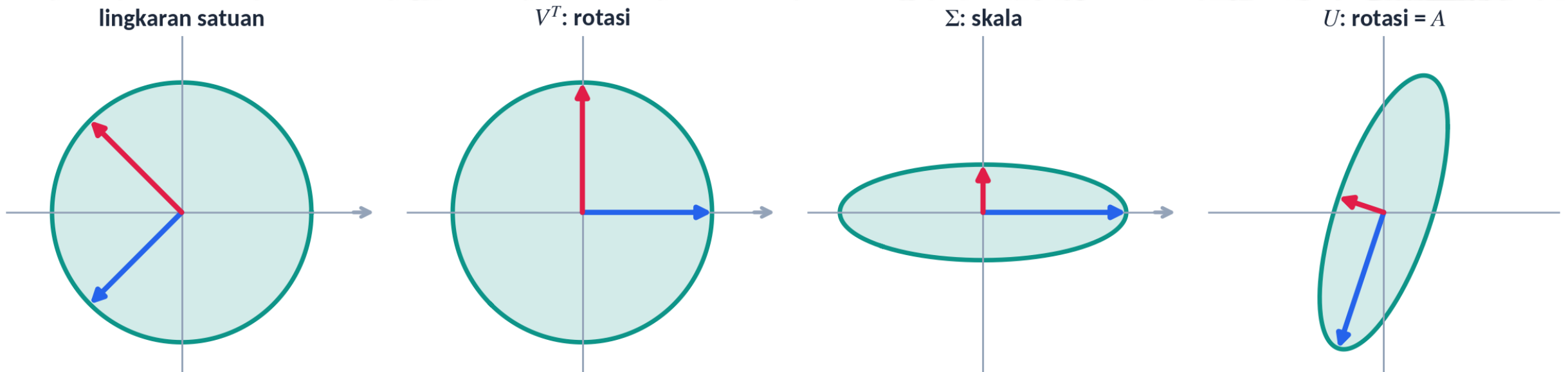
Setiap matriks $A \in \mathbb{R}^{m \times n}$ dapat ditulis sebagai $A = U \Sigma V^T$: rotasi, penskalaan, lalu rotasi.

$$A = U \Sigma V^T$$

V^T rotasi / basis input

Σ skala per arah

U rotasi / basis output



SVD mengatakan: transformasi kompleks dapat dipahami sebagai rotate → scale → rotate. (Skala sumbu tiap panel berbeda.)

Dimensi U, Σ , dan V

Misalkan A berukuran $m \times n$ dengan $m > n$. Ada bentuk penuh (full) dan bentuk hemat (thin).

FULL SVD

$$\begin{matrix} \boxed{A} & = & \boxed{U} & \boxed{\Sigma} & \boxed{V^T} \\ m \times n & & m \times m & m \times n & n \times n \end{matrix}$$

THIN (ECONOMY) SVD, $r = \text{rank}$

$$\begin{matrix} \boxed{A} & = & \boxed{U_r} & \boxed{\Sigma_r} & \boxed{V_r^T} \\ m \times n & & m \times r & r \times r & n \times r \end{matrix}$$

Kolom U

left singular vectors: basis output

Diagonal Σ

$\sigma_1 \geq \sigma_2 \geq \dots \geq 0$: kekuatan tiap arah

Kolom V

right singular vectors: basis input

Insight

Nilai singular mengurutkan “kekuatan” tiap arah; ini memberi ranking alami atas struktur matriks.

Hubungan SVD dengan eigen

SVD dapat dipahami melalui eigen decomposition dari $A^T A$ dan $A A^T$.

$$A^T A = V \Sigma^2 V^T$$

$$A A^T = U \Sigma^2 U^T$$

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma^T (U^T U) \Sigma V^T = V \Sigma^2 V^T \quad \text{karena } U^T U = I$$

Kolom V

eigenvector $A^T A$

Kolom U

eigenvector $A A^T$

σ_i^2

eigenvalue yang
bersesuaian

$\sigma_i \geq 0$

$A^T A$ simetris PSD $\rightarrow$ real,
non-negatif

SVD memperluas ide eigen ke matriks non-persegi dengan memindahkan masalah ke $A^T A$ dan $A A^T$.

Contoh numerik SVD 2×2

Hitung SVD dari $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ langkah demi langkah.

1. Hitung $A^T A$

$$A^T A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

2. Eigenvalue $A^T A$

$$\lambda = 45 \text{ dan } 5 \rightarrow \sigma_1 = \sqrt{45} \approx 6.708, \\ \sigma_2 = \sqrt{5} \approx 2.236$$

3. Eigenvector $\rightarrow V$

$$v_1 = [1, 1]/\sqrt{2}, \quad v_2 = [1, -1]/\sqrt{2}$$

4. $U = A V \Sigma^{-1}$

$$u_1 \approx [0.316, 0.949], \quad u_2 \approx [0.949, -0.316]$$

$$\sigma_1 : \sigma_2$$

$\approx 3 : 1$, arah pertama tiga kali lebih kuat

Verifikasi

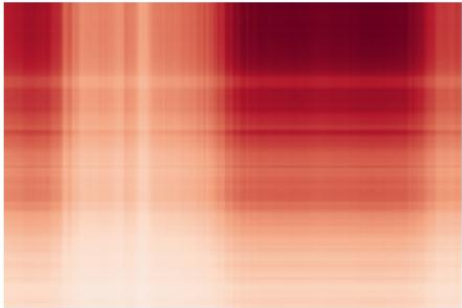
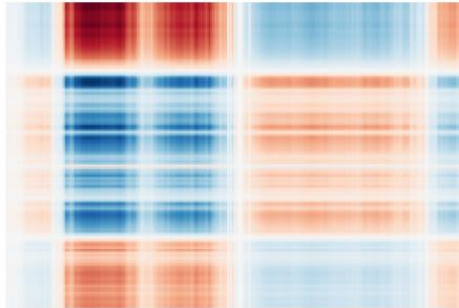
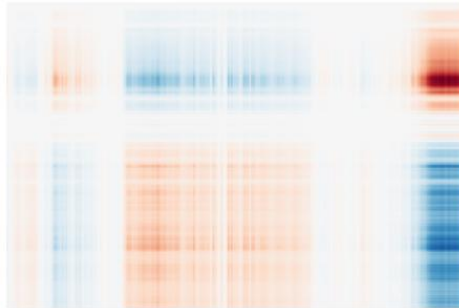
$\|A\|^2 = 9 + 0 + 16 + 25 = 50 = \sigma_1^2 + \sigma_2^2 = 45 + 5$. Jumlah kuadrat semua elemen = jumlah kuadrat nilai singular.

Tanda singular vector tidak unik (v dan -v sama sahinya).

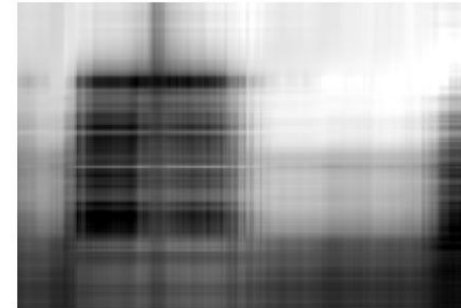
SVD sebagai penjumlahan komponen rank-1

Bentuk lain yang sangat penting: matriks adalah jumlah pola sederhana, diurutkan dari yang paling dominan.

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T$$

 $\sigma_1 u_1 v_1^T$

 $\sigma_2 u_2 v_2^T$

 $\sigma_3 u_3 v_3^T$


jumlah 3 komponen



gambar asli



Insight

Matriks kompleks dapat dibangun sebagai penjumlahan pola sederhana rank-1. Setiap suku adalah hasil kali luar $u_i v_i^T$: satu pola kolom dikali satu pola baris.

rank-1

satu komponen = $u_i v_i^T$

Low-rank approximation dan teorema Eckart–Young

Jika singular value kecil, kontribusi komponennya kecil. Kita potong setelah k komponen.

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T \quad A \approx A_k, \quad k \ll \min(m, n)$$

Original
rank r



Potong k komponen
simpan $\sigma_1 \dots \sigma_k$



Aproksimasi
rank k



Penyimpanan turun

dari mn menjadi $k(m + n + 1)$ angka



Noise sering berkurang

komponen kecil sering didominasi noise



Struktur global bertahan
pola dominan tetap terwakili



Trade-off

fidelity vs compression

Eckart–Young–Mirsky

Untuk semua B dengan rank $\leq k$:
 $\|A - A_k\|^N \leq \|A - B\|^N$ dan galatnya
 $\|A - A_k\|^{N^2} = \sum_{i>k} \sigma_i^2$ (norma spektral: σ_{k+1}). Truncated SVD adalah aproksimasi rank-k terbaik.

Contoh: citra sebagai matriks

Citra grayscale $H \times W$ hanyalah matriks intensitas 200×300 . Simpan hanya top- k singular values.

rank $k = 2$



rank $k = 10$



rank $k = 30$



asli (rank 200)



1.7%

penyimpanan pada $k = 2$ (PSNR 17.4 dB)

8.3%

penyimpanan pada $k = 10$ (PSNR 21.8 dB)

25.1%

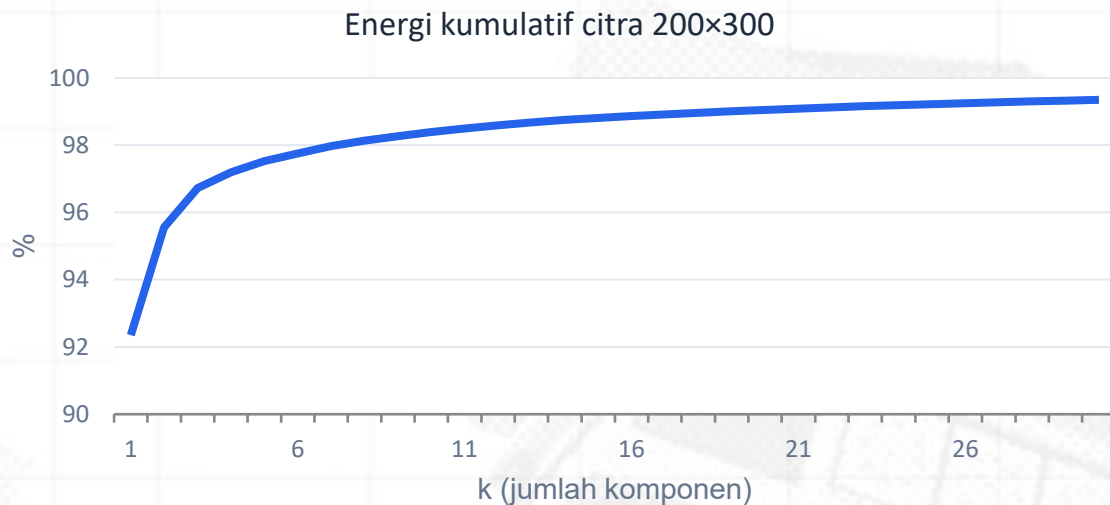
penyimpanan pada $k = 30$ (PSNR 25.7 dB)

Image compression adalah contoh visual paling intuitif untuk memahami *low-rank representation*: pertahankan struktur dominan, abaikan detail berenergi rendah.

Energi yang dipertahankan oleh top-k

Besarnya σ_i^2 dapat dibaca sebagai energi yang dibawa komponen ke-i.

$$\text{Explained energy}(k) = (\sum_{i=1}^k \sigma_i^2) / (\sum_{i=1}^r \sigma_i^2)$$



k	Penyimpanan	Energi	PSNR (dB)
1	0.8%	92.35%	15.0
2	1.7%	95.55%	17.4
5	4.2%	97.52%	19.9
10	8.3%	98.39%	21.8
20	16.7%	99.05%	24.1
30	25.1%	99.35%	25.7
50	41.8%	99.64%	28.3

⚠ σ_1 mencakup ~92% karena kecerahan rata-rata ikut terhitung (SVD tanpa centering $\neq$ PCA). “95% retained” hanya aturan praktis: pilih k berdasarkan tugas hilir.

Rank, redundancy, dan intrinsic dimension

Rank mengukur banyaknya arah linear independen yang benar-benar diperlukan.

1000

dimensi nominal
(kolom fitur)

$\approx$ 40

rank efektif linear

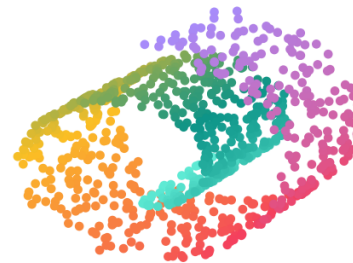
2

dimensi intrinsik
(swiss roll)

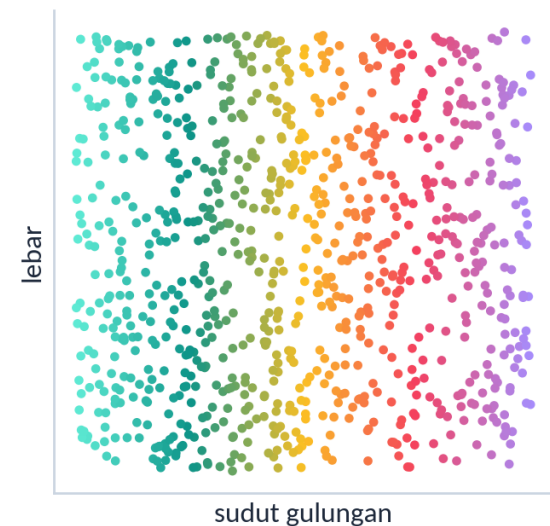
$\text{rank}(A) = \text{jumlah } \sigma_i > 0$

- Secara nominal 1000 dimensi, tetapi struktur linear efektif mungkin jauh lebih kecil
- *Exact rank* sensitif terhadap noise
- *Effective rank* melihat spektrum singular value
- *Intrinsic dimension* dapat lebih rendah lagi jika struktur data nonlinear

ruang 3D (nominal)



koordinat intrinsik: 2D



Latent dimension bukan sekadar jumlah fitur; ia mencoba menangkap derajat kebebasan yang benar-benar membentuk data.

SVD dalam praktik: *pseudo-inverse*, *ridge*, dan *conditioning*

SVD adalah pisau serbaguna. Tiga penggunaan yang langsung terhubung dengan P3.



Pseudo-inverse

$$A^+ = V \Sigma^+ U^T$$

solusi least squares $w = A^+y$ (norma minimum)



Condition number

$$\kappa(A) = \sigma_1 / \sigma_r$$

κ besar $\rightarrow$ problem sensitif terhadap noise

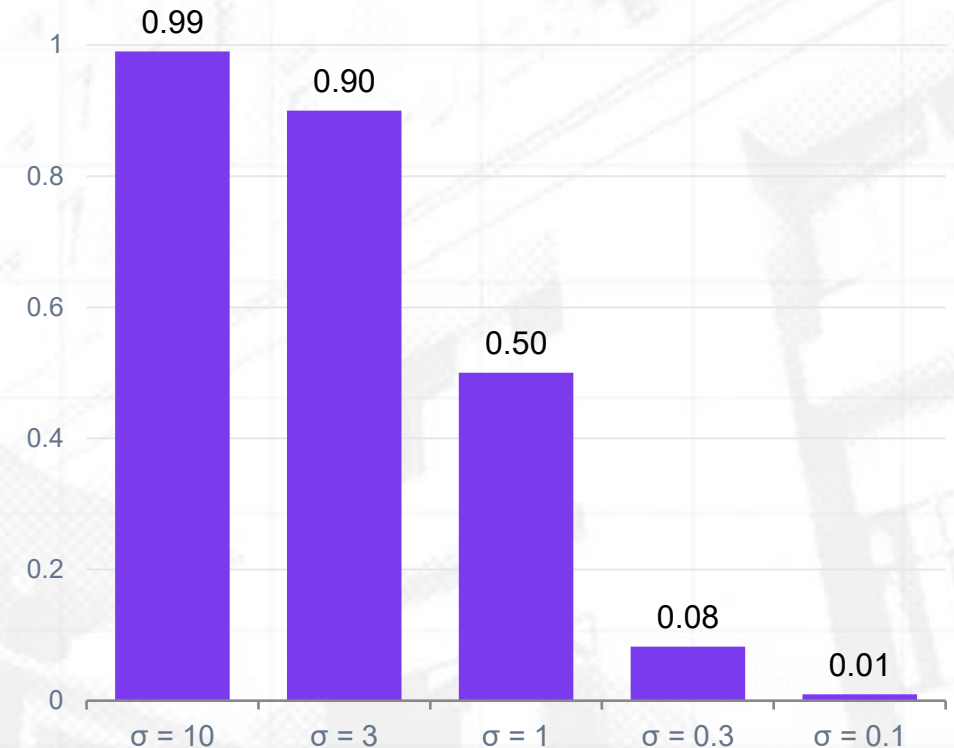


Ridge (P3)

$$f_i = \sigma_i^2 / (\sigma_i^2 + \lambda)$$

menekan arah dengan σ kecil

Faktor penyusutan ridge ($\lambda = 1$)



03

PART III · PCA

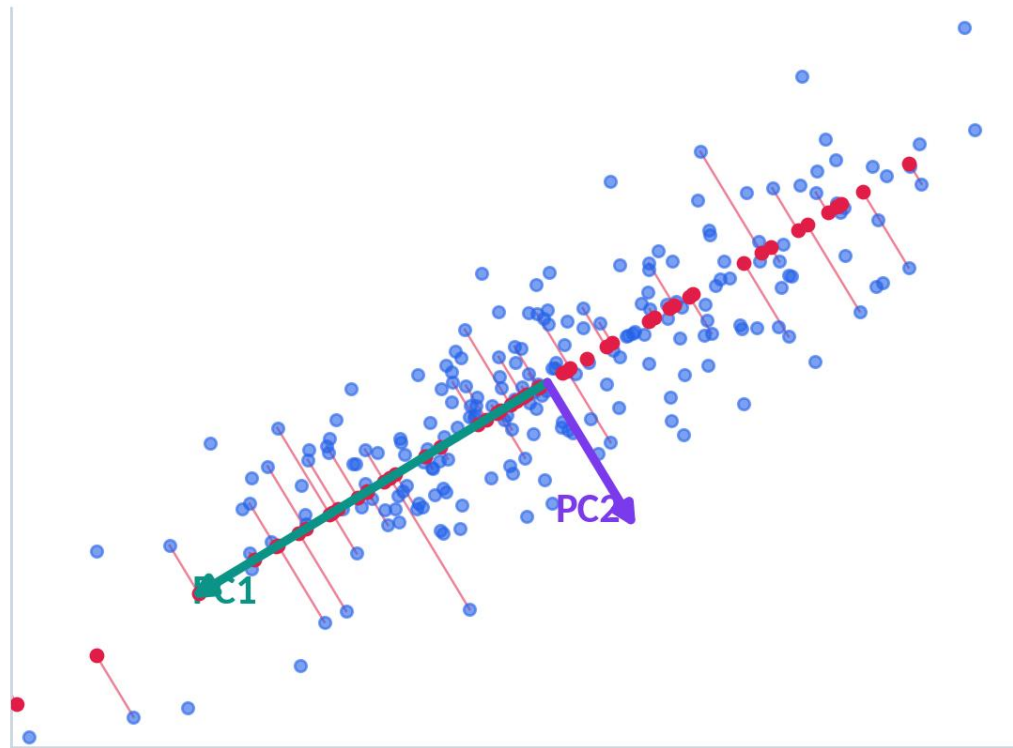
PCA: mencari arah variasi terbesar

$$Z = X^c W_k$$

- 1 Intuisi dari scatter plot
- 2 Center → covariance → eigen
- 3 Mengapa PC1 memaksimalkan variance
- 4 Proyeksi, rekonstruksi, dan hubungan dengan SVD
- 5 Loadings dan memilih k
- 6 Covariance vs correlation
- 7 Keterbatasan dan alternatif nonlinear

Intuisi PCA dari *scatter plot*

Data dua dimensi sering berada dekat sebuah garis. PCA menemukan garis itu.



Garis merah = galat proyeksi yang diminimalkan.

2D data

awan titik miring

Project onto PC1

jatuhkan tiap titik ke garis

Simpan koordinat

satu bilangan per sampel

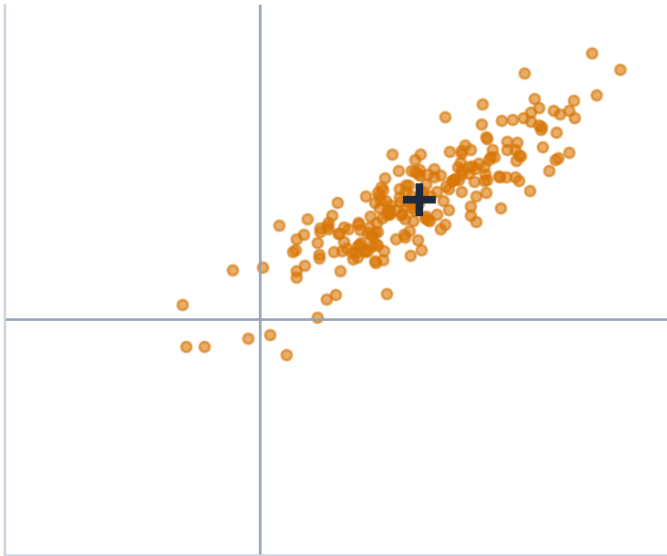
$$z = X w_1$$

PCA mempertahankan arah dengan variasi terbesar dan membuang arah dengan variasi kecil.

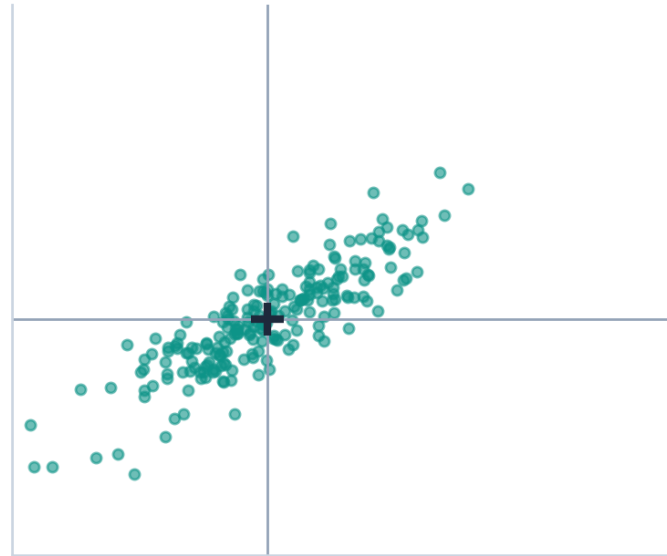
Langkah 1: *center the data*

PCA klasik mengasumsikan data telah dikurangi mean per fitur.

data mentah: pusat di (4.1, 3.0)



setelah $X_c = X - 1\mu^T$: pusat di 0



$$\mu = (1/n) \sum_i x_i$$

$$X^c = X - 1 \mu^T$$

centroid $\mu \rightarrow$ dipindahkan ke titik asal

Centering menggeser origin ke pusat data sehingga covariance benar-benar mengukur variasi di sekitar mean.

Tanpa centering

PCA dapat menangkap lokasi mean, bukan pola variasi.

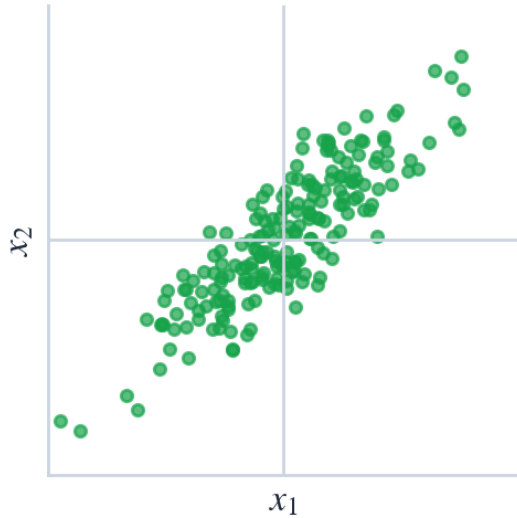
Skala berbeda?

Standardization mungkin diperlukan sebelum PCA

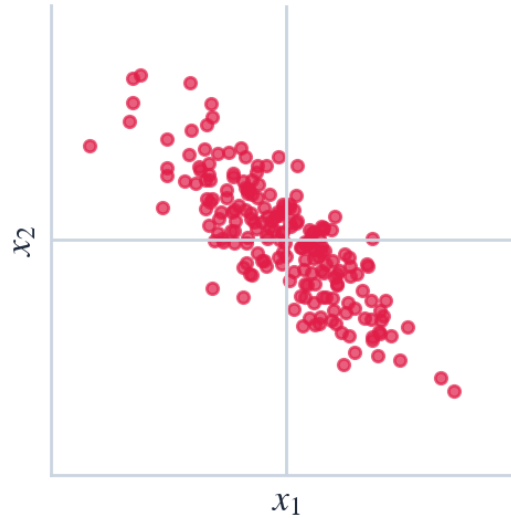
Langkah 2: *covariance matrix*

Covariance mengukur bagaimana dua fitur berubah bersama.

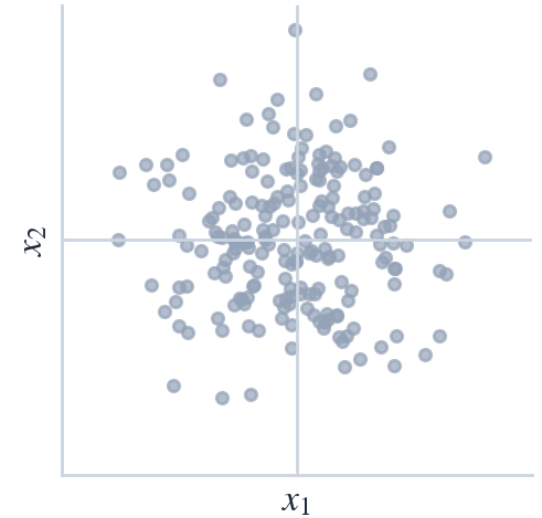
cov > 0: miring naik



cov < 0: miring turun



cov ≈ 0: tanpa kemiringan



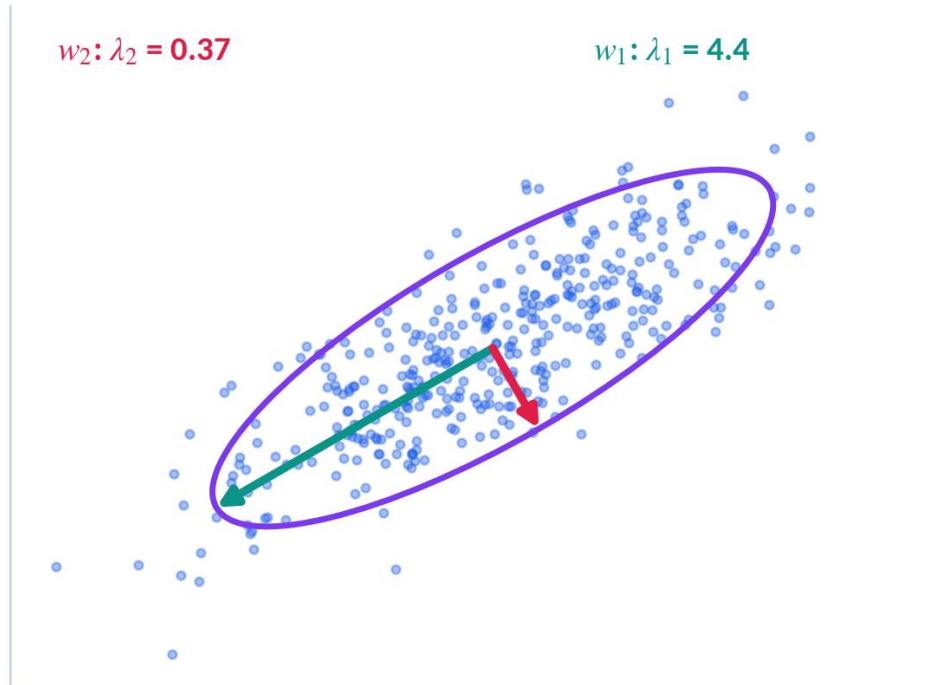
$$C = (1/(n-1)) X^{cT} X^c$$

$$C = \begin{bmatrix} \text{var}(x_1) & \text{cov}(x_1, x_2) \\ \text{cov}(x_2, x_1) & \text{var}(x_2) \end{bmatrix}$$

Covariance matrix adalah ringkasan geometri linear data; eigenvector-nya menjadi principal directions.

Langkah 3: *eigen decomposition covariance*

Karena C simetris, eigenvector-nya ortonormal. Elips mencerminkan sebaran data.



Elips = kontur 2σ ; panah = eigenvector $\times \sqrt{\lambda}$.

Eigenvalue PCA bukan angka abstrak: ia persis *variance* data setelah diproyeksikan pada *eigenvector* terkait.

$$C w_j = \lambda_j w_j$$

w_1

arah variance terbesar

w_2

arah terbesar berikutnya (ortogonal)

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$$

$$\text{Var}(X^c w_j) = \lambda_j$$

Mengapa PC1 memaksimalkan variance?

Cari vektor satuan w yang membuat proyeksi $z = X^c w$ memiliki variance maksimum.

1 Problem

$$\text{maximize } w^T C w \quad \text{subject to } w^T w = 1$$

2 Lagrangian

$$L(w, \lambda) = w^T C w - \lambda (w^T w - 1)$$

3 Stasioner

$$\partial L / \partial w = 2Cw - 2\lambda w = 0 \Rightarrow Cw = \lambda w$$

4 Nilai optimum

$$w^T C w = \lambda w^T w = \lambda \rightarrow \text{pilih } \lambda \text{ terbesar}$$

Hasil

Maksimum dicapai oleh eigenvector dengan eigenvalue terbesar. PC2 dicari dengan kendala tambahan $w_2^T w_1 = 0$, dan seterusnya.

PCA lahir dari problem optimasi; eigenvector muncul sebagai kondisi optimum.

Sudut pandang setara

Memaksimalkan variance = meminimalkan galat proyeksi.

Proyeksi ke ruang PCA

Jika W_k berisi k principal components sebagai kolom, setiap sampel dipetakan ke k koordinat.

$$Z = X^c W_k$$

d fitur $\rightarrow k$ dimensi laten

$$\begin{array}{ccc}
 \boxed{X^c} & \cdot & \boxed{W_k} = \boxed{Z} \\
 n \times d & & d \times k \quad n \times k
 \end{array}$$

- Setiap baris Z adalah representasi baru dari satu sampel
- Dimensi turun dari d ke k
- Koordinat baru tidak lagi identik dengan fitur asli

PCA menghasilkan embedding linear: setiap objek dipetakan menjadi koordinat pada *principal-component space*.

Untuk data baru: pakai μ dan W_k dari data training, jangan dihitung ulang.

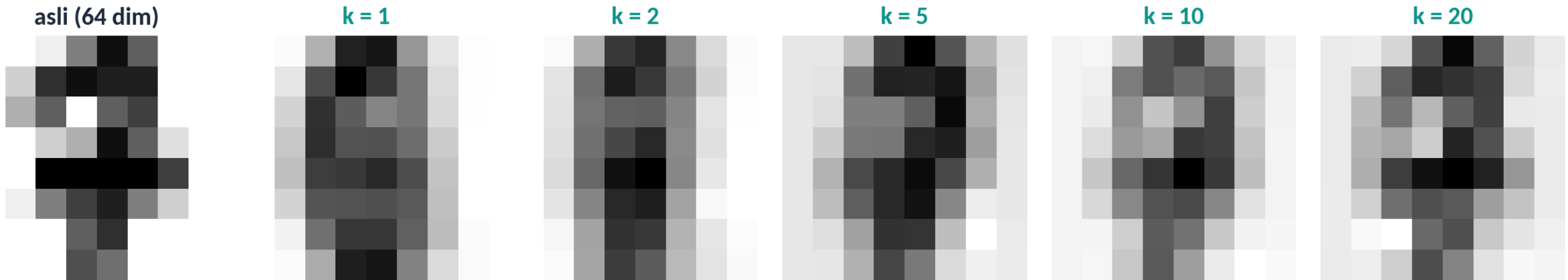
Reconstruction: apa yang hilang?

Representasi laten dapat diproyeksikan kembali ke ruang asli. Contoh: digit 8×8 (64 dimensi).

$$\hat{X} = Z W_k^T + \mu$$

$$\text{galat} = \|X - \hat{X}\|^{N^2}$$

$$\propto \sum_{i>k} \lambda_i$$



k	1	2	5	10	20
MSE / piksel	15.98	13.42	8.54	4.91	1.98

Semakin kecil k, semakin kompak representasi, tetapi semakin besar informasi yang hilang.

PCA dan SVD sebenarnya sangat dekat

Untuk centered matrix $X^c = U \Sigma V^T$, seluruh PCA terbaca langsung dari SVD.

$$X^c = U \Sigma V^T$$

$$C = X^{cT} X^c / (n-1) = V [\Sigma^2 / (n-1)] V^T$$

Dalam PCA	Terbaca dari SVD	Catatan
Principal directions W	kolom V	urutan sesuai σ_i menurun
Eigenvalue λ_i	$\sigma_i^2 / (n - 1)$	variance per komponen
Scores $Z = X^c W_k$	$U_k \Sigma_k$	koordinat sampel
Explained variance ratio	$\sigma_i^2 / \sum \sigma_j^2$	untuk memilih k



Praktik

PCA sering dihitung dengan SVD karena lebih stabil dan efisien daripada membentuk covariance matrix secara eksplisit.



Eigen, SVD, dan PCA bukan tiga pulau terpisah: mereka adalah tiga sudut pandang atas struktur linear yang sama.

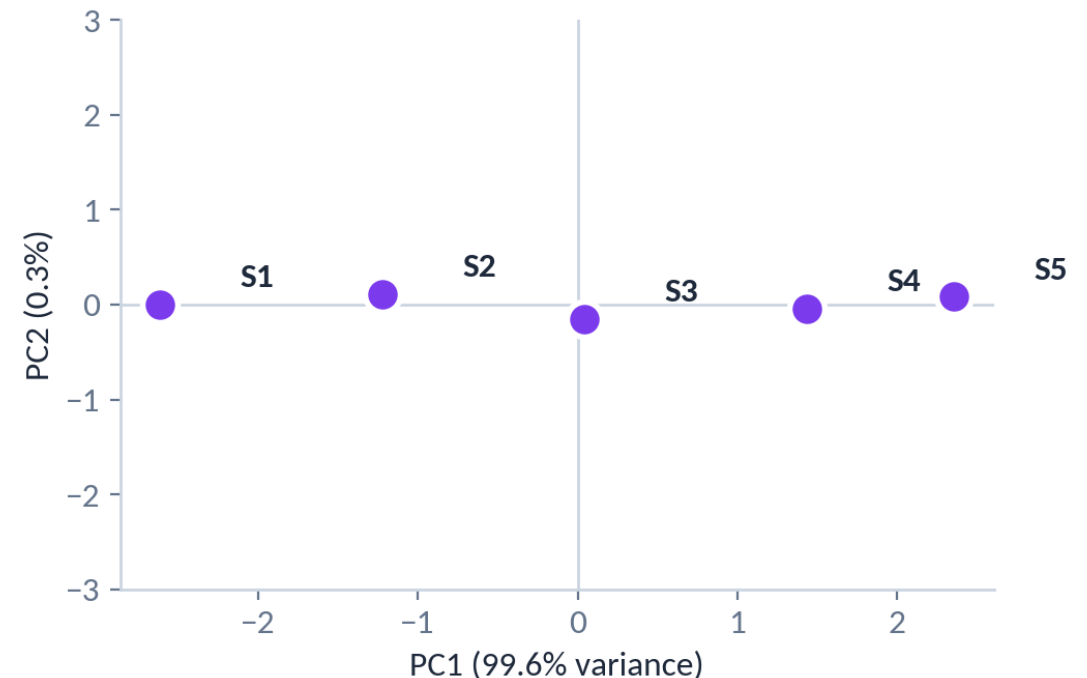
Contoh kasus PCA: 4 fitur → 2 latent dimensions

Fitur mahasiswa: jam belajar, latihan soal, nilai kuis, nilai tugas. Data distandardisasi (z-score) sebelum PCA.

	jam	latihan	kuis	tugas
S1	2	3	55	60
S2	4	5	65	68
S3	6	6	75	78
S4	8	8	85	86
S5	10	9	92	90

↓ center + standardize + PCA

	PC1	PC2
S1	-2.61	0.00
S2	-1.22	0.11
S3	0.04	-0.15
S4	1.43	-0.04
S5	2.36	0.08

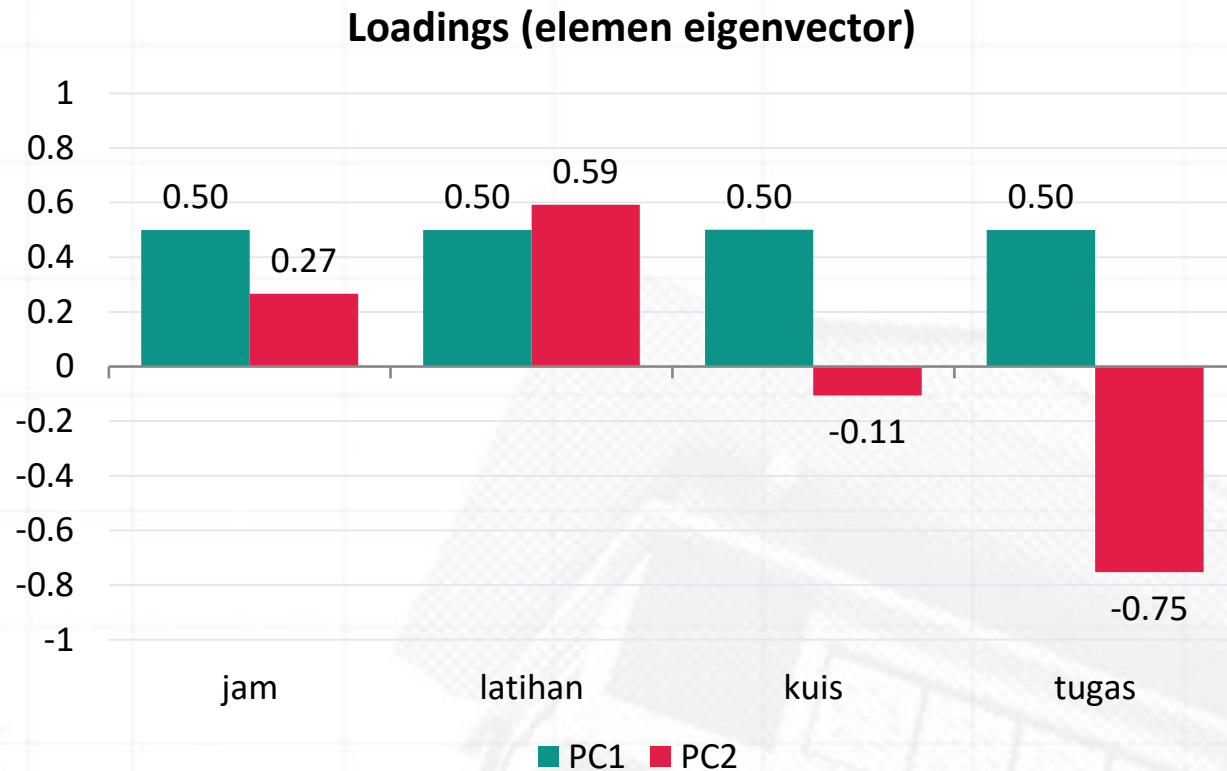


PC1 99.6% variance: “overall engagement”

Nama laten tidak diberikan otomatis oleh PCA. Interpretasi butuh loading, konteks domain, dan kehati-hatian.

PCA loadings: fitur mana membentuk komponen?

Elemen *eigenvector* menunjukkan kontribusi tiap fitur pada *principal component*.



PC1

bobot positif hampir sama pada semua fitur → pola umum (“engagement”)

PC2

membedakan effort (jam, latihan) vs outcome (kuis, tugas), tetapi varian kecil

Tanda

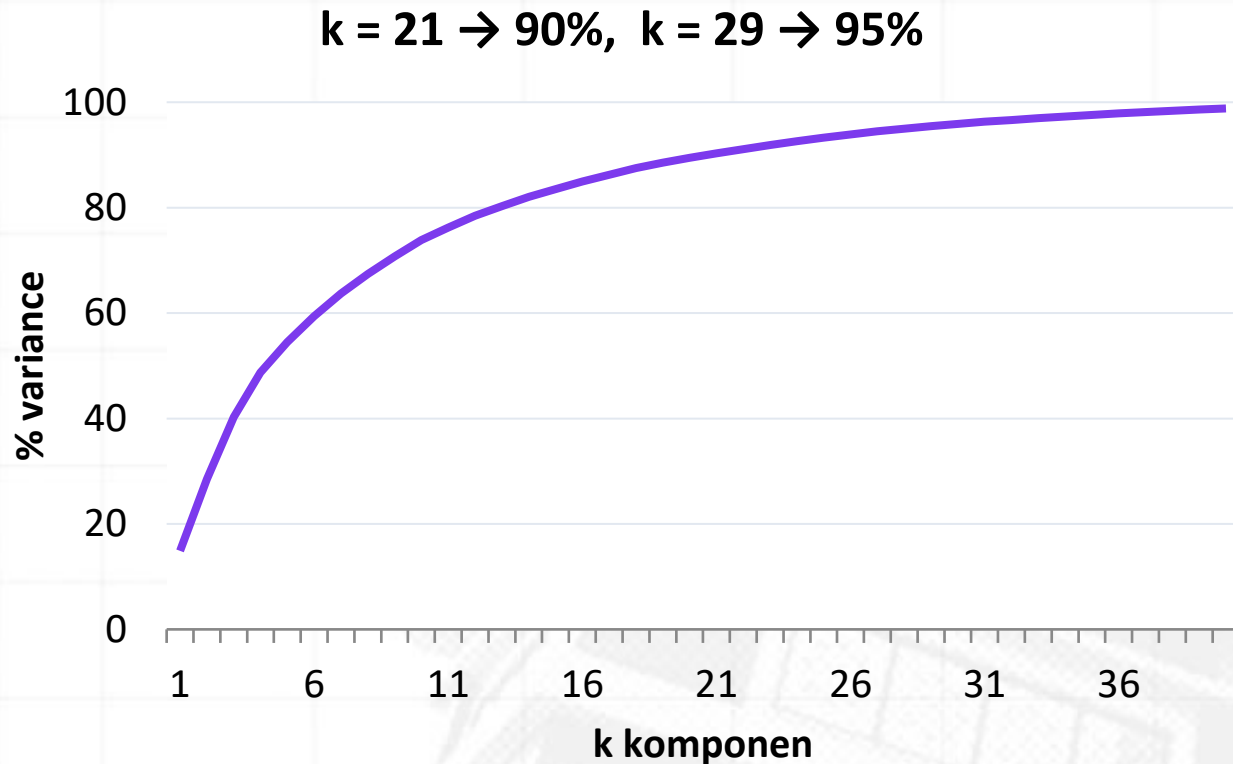
eigenvector boleh dibalik ($w \rightarrow -w$) tanpa mengubah subspace

Perhatikan: tanda w_2 dan besar loading bergantung pada data; ini contoh, bukan hukum.

Interpretasi komponen sebaiknya berbasis pola loading, bukan memberi label semantik secara bebas.

Memilih k: *scree plot* dan *explained variance*

Contoh nyata: dataset digit 8×8 (64 dimensi, 1797 sampel).



Ambang kumulatif
mis. 90–95% variance

Elbow / scree
titik siku pada plot λ_i

Kaiser (PCA korelasi)
pertahankan $\lambda_i > 1$

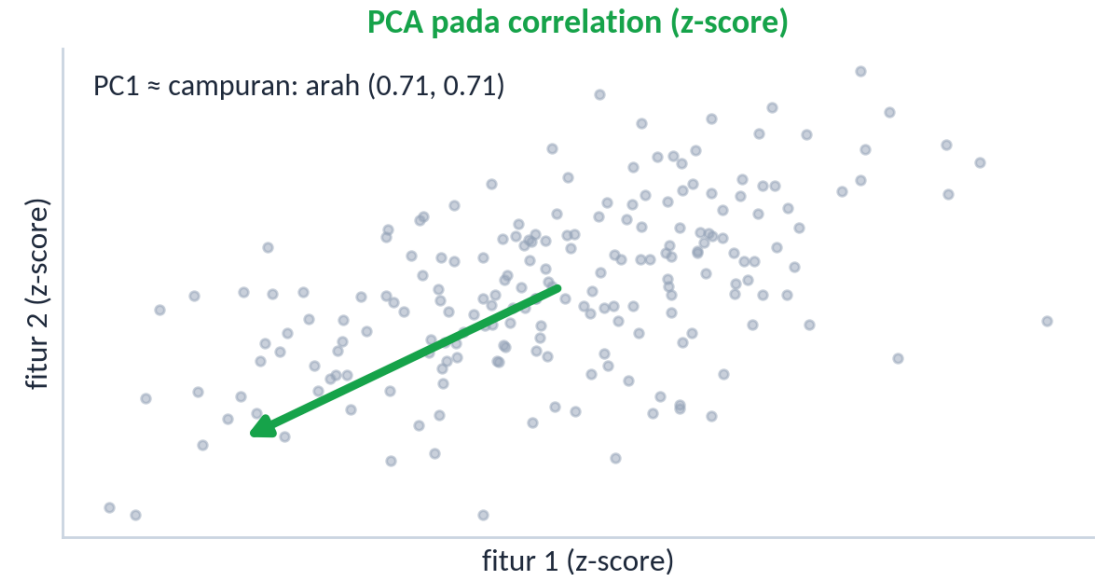
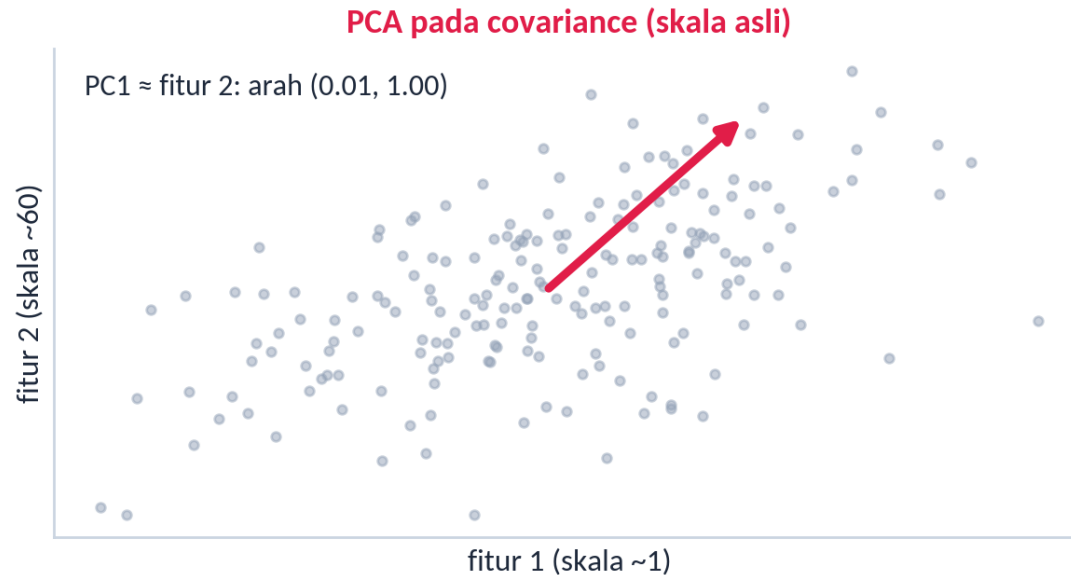
Cross-validation
pilih k terbaik untuk tugas hilir

Parallel analysis
bandingkan dengan spektrum data acak

Untuk tugas prediktif, k terbaik ditentukan oleh performa hilir, bukan hanya oleh variance.

PCA: *covariance* atau *correlation*?

Keputusan preprocessing dapat mengubah principal components secara drastis.



Covariance PCA (skala asli)

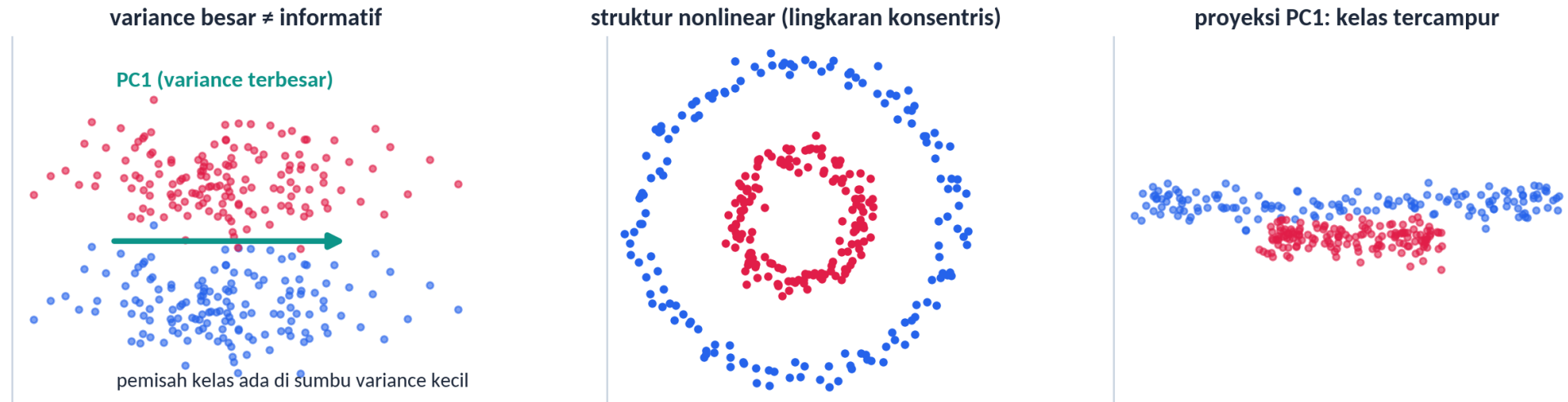
Fitur dengan variance besar dominan.
Cocok jika unit/skalanya sebanding.

Correlation PCA (z-score)

$z_j = (x_j - \mu_j)/s_j$, semua fitur variance ≈ 1 .
Cocok jika unit berbeda jauh.

Keterbatasan PCA

PCA sangat kuat, tetapi hanya melihat struktur linear dan variance.



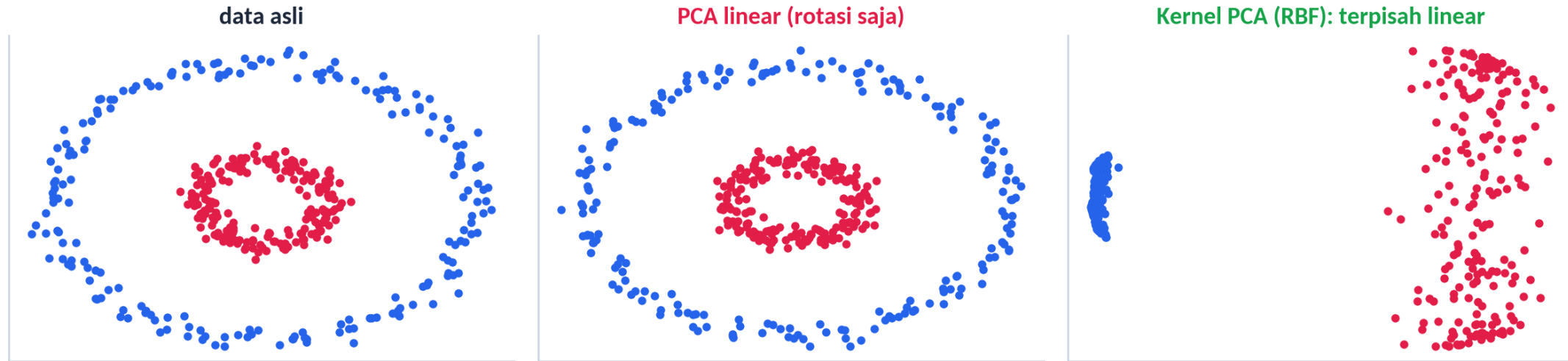
- *Variance* besar tidak selalu berarti informasi penting untuk target
- PCA unsupervised: label y tidak digunakan
- *Manifold* melengkung tidak ditangkap optimal
- Komponen dapat sulit diinterpretasikan

- *Outlier* dapat mengubah covariance dan arah PC
- Preprocessing sebelum split $\rightarrow$ data leakage
- Sensitif terhadap skala fitur

Gunakan PCA sebagai alat representasi, bukan asumsi bahwa “variance terbesar = makna terbesar”.

Melampaui PCA: representasi nonlinear

Kernel PCA memisahkan lingkaran konsentris yang tidak bisa dipisahkan oleh PCA linear.



Metode	Ide	Kelebihan	Hati-hati
Kernel PCA	PCA di ruang fitur implisit (kernel)	nonlinear, ada solusi tertutup	butuh kernel dan gamma; skala $O(n^2)$
t-SNE / UMAP	menjaga tetangga lokal saat memvisualisasi	cluster tampak jelas di 2D	jarak global dan ukuran cluster menyesatkan
Autoencoder	encoder-decoder nonlinear (Part V)	fleksibel, skalabel	butuh training; sulit dianalisis

04

Embedding: representasi yang dipelajari

$$f : \mathcal{X} \rightarrow \mathbb{R}^d$$

- 1 Apa itu embedding? One-hot $\rightarrow$ dense
- 2 Embedding matrix sebagai parameter
- 3 Contrastive learning dan word2vec
- 4 Case: citra, teks, cross-modal
- 5 PCA vs learned embedding; autoencoder
- 6 Latent space bukan selalu semantik
- 7 Mengevaluasi representation

Apa itu embedding?

Sebuah fungsi yang memetakan objek ke vektor berdimensi tetap.



$$f : \mathcal{X} \rightarrow \mathbb{R}^d$$

A kata

d dokumen

c citra

a audio

n node
graph

p pasien

p produk

Embedding yang baik

Membuat relasi yang relevan untuk task menjadi mudah dibaca secara geometris (jarak kecil = mirip).

Embedding bukan “fitur ajaib”: maknanya berasal dari objective, data training, arsitektur, dan evaluasi.

Dari *one-hot* ke *dense embedding*

Representasi one-hot besar dan tidak menyatakan similarity.

one-hot: cosine antar token

cat	1.0	0.0	0.0	0.0	0.0
dog	0.0	1.0	0.0	0.0	0.0
car	0.0	0.0	1.0	0.0	0.0
truck	0.0	0.0	0.0	1.0	0.0
banana	0.0	0.0	0.0	0.0	1.0
	cat	dog	car	truck	banana

dense embedding: cosine

cat	1.0	1.0	-0.5	-0.6	-0.2
dog	1.0	1.0	-0.4	-0.5	-0.4
car	-0.5	-0.4	1.0	1.0	-0.7
truck	-0.6	-0.5	1.0	1.0	-0.6
banana	-0.2	-0.4	-0.7	-0.6	1.0
	cat	dog	car	truck	banana

$\text{one-hot}(\text{cat}) = [0, 0, 1, 0, 0, \dots]$

$\text{one-hot}(\text{dog}) = [0, 0, 0, 1, 0, \dots]$

Masalah one-hot

Dimensi = ukuran kosakata (bisa jutaan) dan $\cos = 0$ untuk semua token berbeda: tidak ada kedekatan makna.

lookup
encoder

$[0.2, -0.7, \dots]$
dense vector

Dense embedding memungkinkan similarity kontinu: dua objek berbeda dapat dekat jika training objective mendorongnya.

Embedding matrix sebagai parameter

Untuk vocabulary V dan dimensi d , embedding lookup adalah memilih satu baris dari matriks $E \in \mathbb{R}^{|V| \times d}$.

token	d_1	d_2	d_3	d_4
the	-0.1	0.3	0.5	0.0
cat	0.2	-0.7	0.4	0.9
sat	0.6	0.1	-0.5	0.2
on	-0.3	0.4	0.1	-0.2
mat	0.5	-0.4	0.3	0.7
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

E ($|V|$ baris, satu vektor per token)

$$z = E^T x \quad (x = \text{one-hot})$$

≈ 38 juta

1 baris

parameter untuk $|V| = 50.000$, $d = 768$

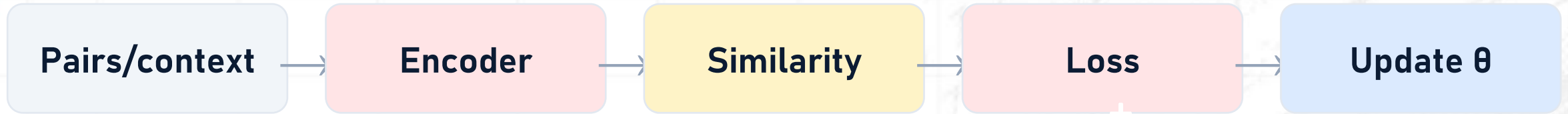
yang dipilih tiap token (lookup)

- E dipelajari melalui gradient descent bersama objective model
- Hanya baris token yang muncul di batch yang menerima gradien

Embedding klasik pun kembali ke aljabar linear: matriks parameter yang dipelajari.

Bagaimana embedding memperoleh makna?

Makna geometris muncul dari objective, bukan dari dimensinya sendiri.



$$L_i = -\log \frac{\exp\left(\frac{\text{sim}(z_i, z_i^+)}{\tau}\right)}{\sum_j \exp\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)}$$

Positif

z^+ : pasangan yang seharusnya mirip (dua augmentasi, gambar-caption)

Negatif

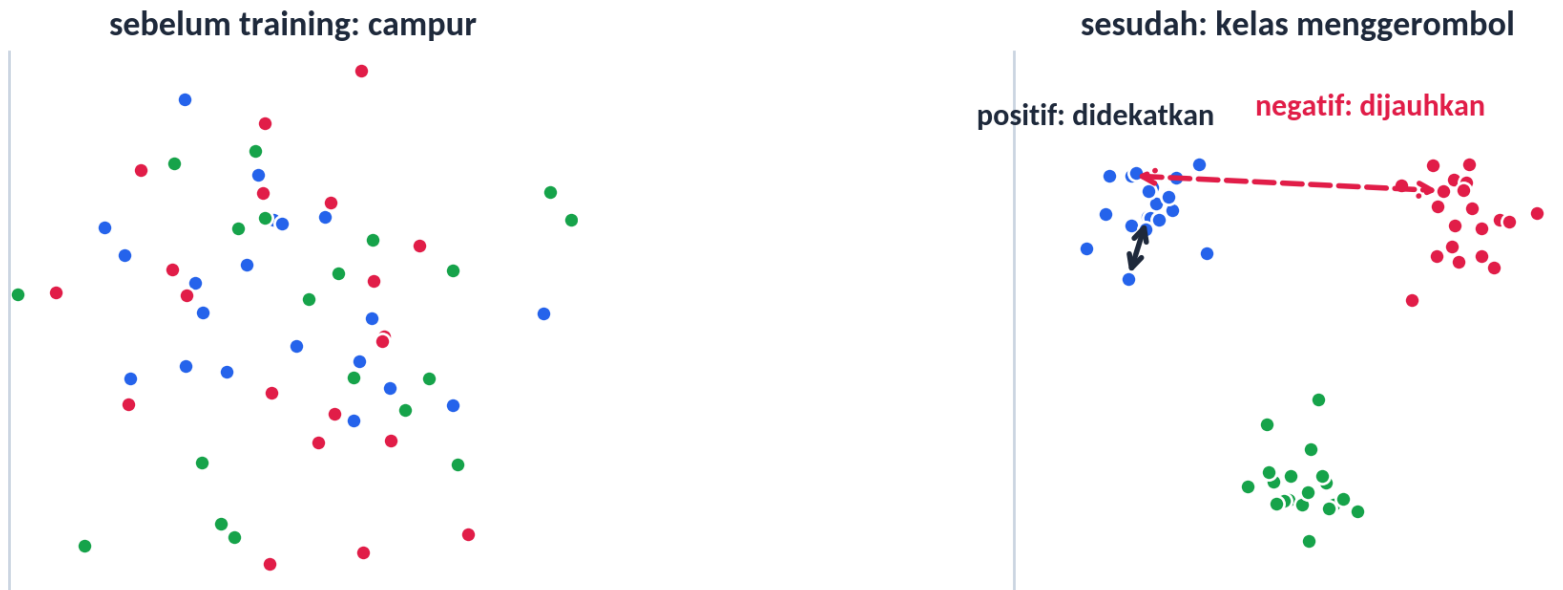
z_j : sampel lain dalam batch; didorong menjauh

Temperature τ mengatur ketajaman; τ kecil menekankan negatif sulit

Jika objective berubah, geometri embedding juga berubah. “Semantik” selalu terkait dengan sinyal *training*.

Contrastive learning secara geometris

Positif didekatkan; negatif dijauhkan dalam latent space. Simulasi ilustratif.



cluster A / B

kelas yang sama menggerombol

loss shapes the geometry

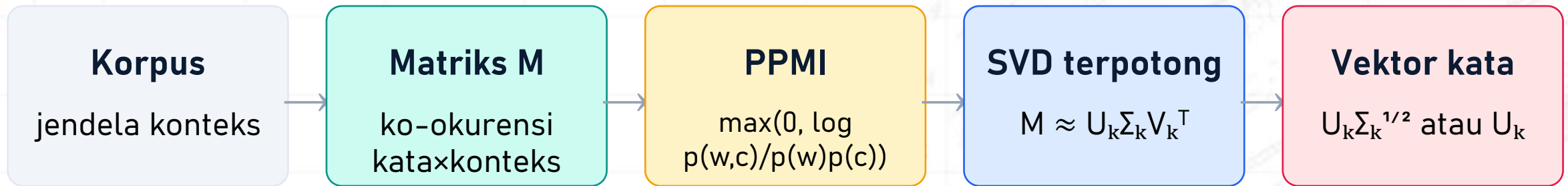
gradien menarik dan mendorong titik

useful latent space

tetangga terdekat = relevan

Word2vec dan hubungannya dengan faktorisasi matriks

Hipotesis distribusional: kata dimaknai oleh konteks tempat ia muncul. Ini menghubungkan embedding dengan SVD.



$$w^T c \approx \text{PMI}(w, c) - \log k$$

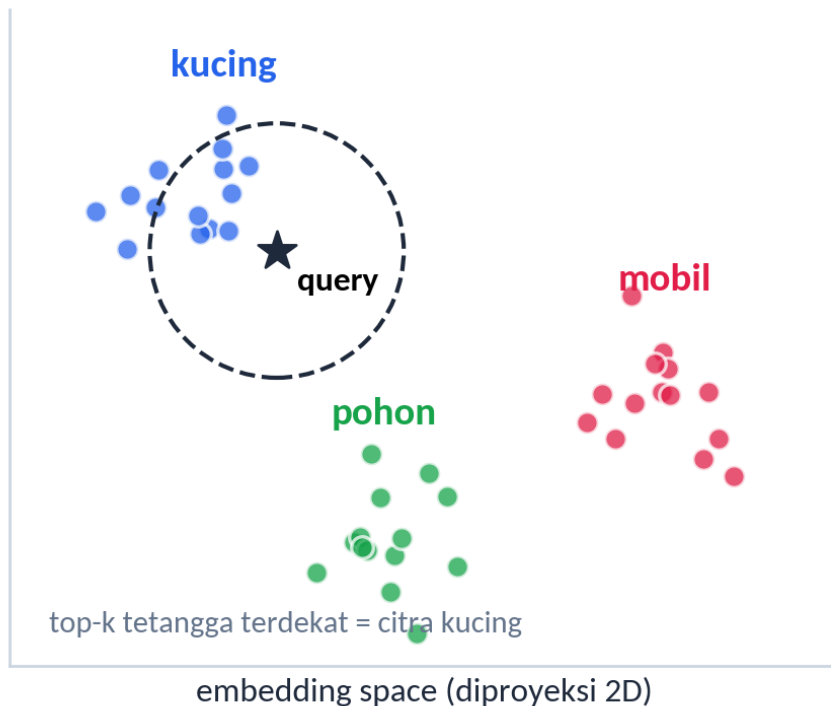
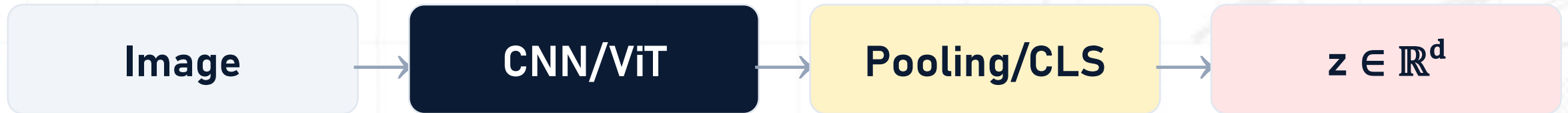
dengan:

- w = vektor target word
- c = vektor context word
- $w^T c$ = dot product kedua vektor
- $\text{PMI}(w, c)$ = seberapa kuat hubungan *word* w dengan *context* c
- k = jumlah negative samples

- Word2Vec belajar vektor kata sehingga dot product-nya mencerminkan kekuatan hubungan kata-konteks.
- Secara konseptual:
 Hubungan kata-konteks $\Rightarrow$ PMI
 $\Rightarrow$ Dot Product $\Rightarrow$ Embedding
- Jadi Word2Vec bukan sekadar “mengubah kata menjadi vektor”. Proses training-nya membuat posisi vektor merepresentasikan pola kemunculan kata dalam konteks.

Case: *image embedding*

Encoder CNN/ViT mengubah citra menjadi vektor berdimensi tetap.



Kucing dekat kucing
jarak kecil antar citra sejenis

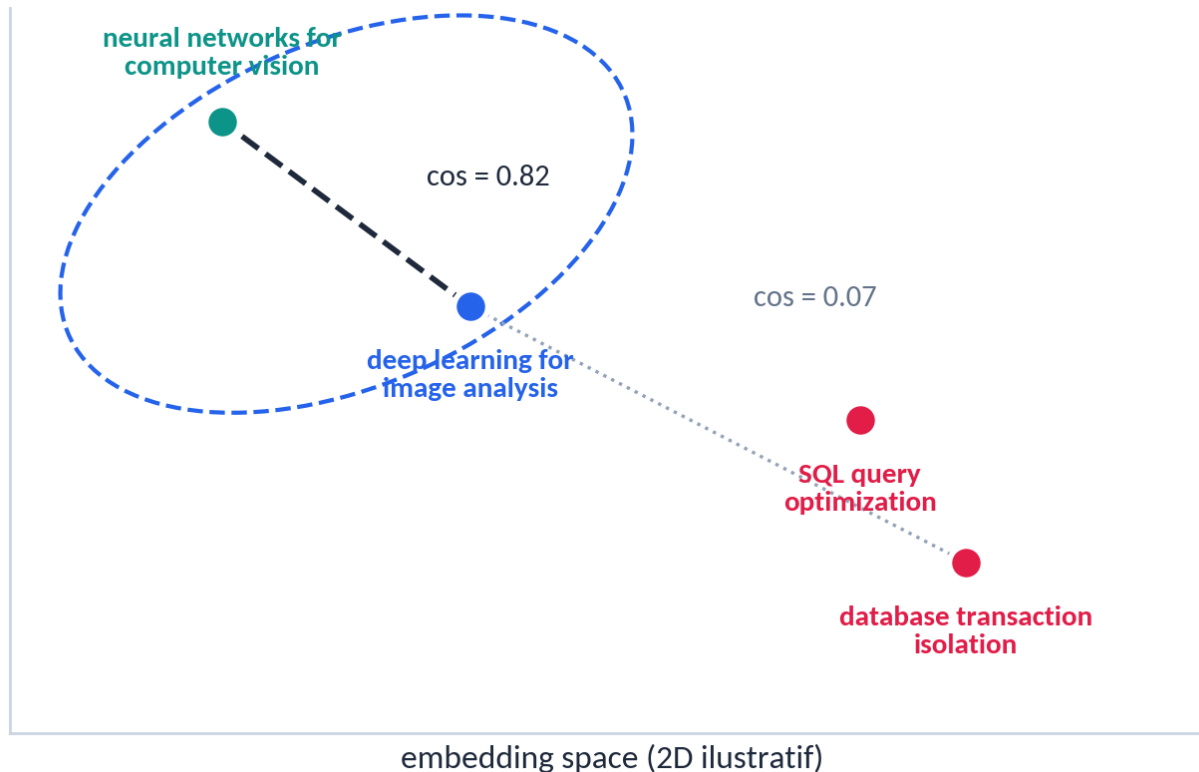
Query → tetangga
ambil top-k terdekat di embedding space

Bukan piksel
objective menjaga neighborhood, bukan rekonstruksi

Untuk retrieval, yang penting bukan merekonstruksi piksel sempurna, tetapi menjaga neighborhood yang relevan.

Case: *text embedding*

Sentence/document encoder memetakan teks dengan panjang berbeda ke vektor berdimensi tetap.



Ruang 2D ilustratif; nilai cosine contoh.

“deep learning for image analysis”

“neural networks for computer vision”

“database transaction isolation”

Makna, bukan kata

Kedekatan bisa terjadi walau tidak ada kata yang sama.

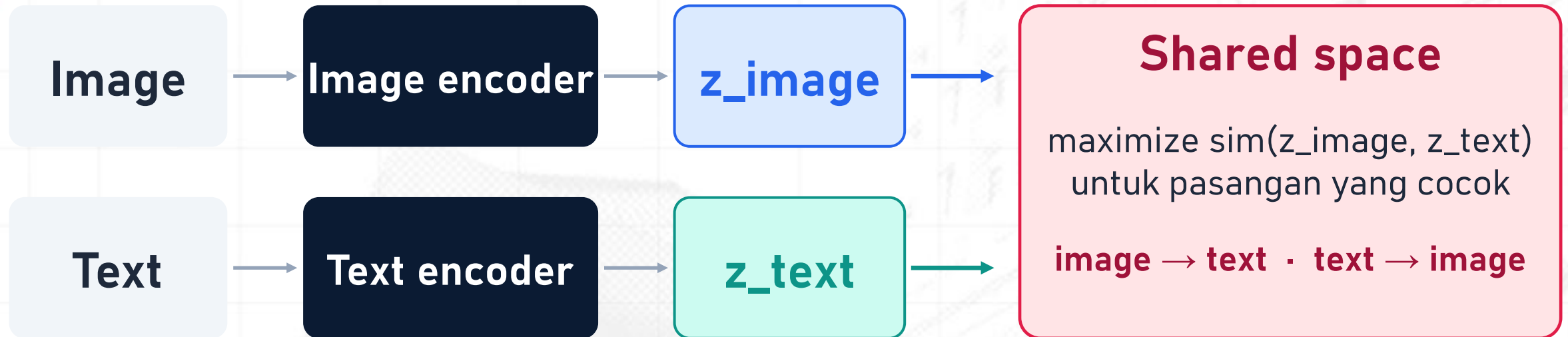
Semantic search

Query di-embed lalu dicari tetangga terdekatnya.

Similarity di embedding space dapat menangkap kedekatan makna meskipun kata-kata permukaannya berbeda.

Cross-modal embedding

Citra dan teks dapat diproyeksikan ke ruang yang sama, misalnya lewat contrastive pretraining pasangan (gambar, caption).



- Memungkinkan image→text dan text→image retrieval
- Alignment membutuhkan paired data atau objective yang menghubungkan modalitas
- Dimensi sama tidak cukup; ruang harus dilatih agar aligned



“Same dimension” ≠ “same semantic space”. Alignment adalah masalah pembelajaran, bukan sekadar reshape/projection.

Linear projection saat dimensi berbeda

Fitur citra 30D dan teks 1000D dapat diproyeksikan ke ruang bersama berdimensi d.



7.680

parameter W_{img} ($d = 256$)

256.000

parameter W_{txt} ($d = 256$)

Projection matrices harus dipelajari dengan objective alignment; nilainya bukan dipilih manual secara arbitrer.

PCA vs learned embedding

Keduanya menghasilkan latent vectors, tetapi tujuannya berbeda.

PCA

- Linear, closed-form / SVD
- Memaksimalkan variance
- Tidak menggunakan label/task
- Cepat dan relatif interpretable

Learned embedding

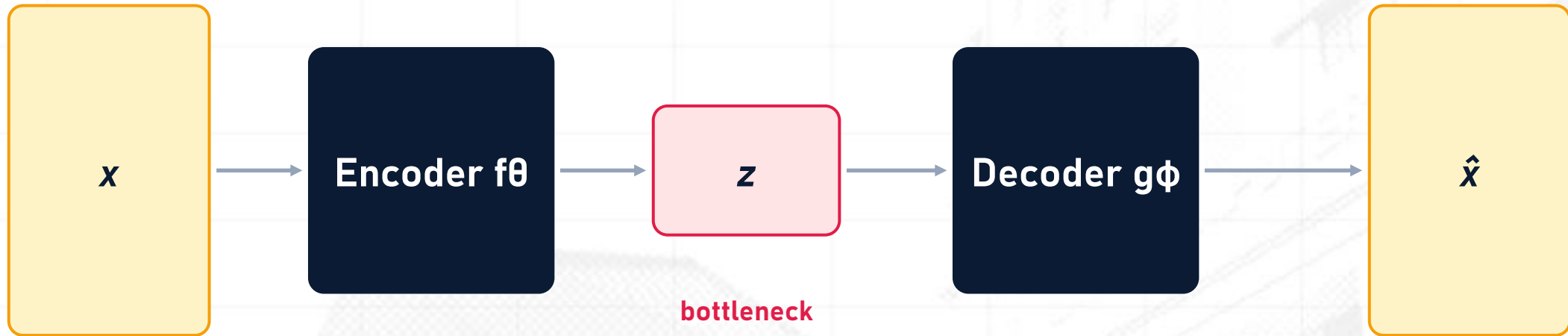
- Biasanya encoder nonlinear
- Objective task / contrastive / reconstruction
- Dapat memakai label atau paired data
- Lebih fleksibel tetapi data-dependent

Aspek	PCA	Learned embedding
Bentuk pemetaan	linear	nonlinear
Objective	variance / galat rekonstruksi linear	ditentukan oleh task
Butuh training?	tidak (solusi tertutup)	ya (gradient descent)
Data baru	proyeksi langsung	forward pass encoder

PCA bertanya “arah mana paling bervariasi?”; learned embedding bertanya “geometri apa yang membantu objective saya?”

Autoencoder: *latent representation* lewat reconstruction

Encoder dan decoder dipelajari bersama; bottleneck z memaksa kompresi.



$$\hat{x} = g_\phi(f_\theta(x))$$

$$L_{\text{rec}} = \|x - \hat{x}\|^2$$

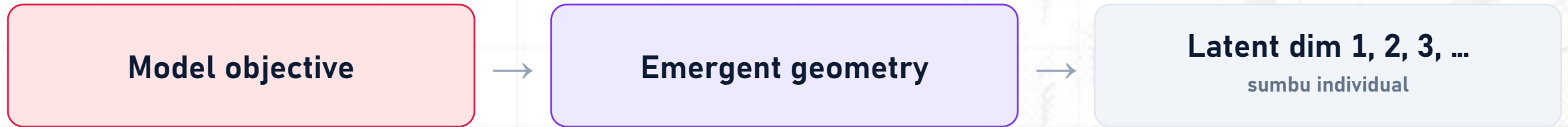
Ini adalah **reconstruction loss**, yang umum digunakan pada **Autoencoder**.

- Encoder memampatkan data → Decoder mencoba mengembalikannya → Loss mengukur kesalahan rekonstruksi.
- reconstruction loss berusaha agar **informasi penting dari data tetap dipertahankan dalam representasi laten**.

Reconstruction menjaga informasi untuk membangun ulang input; objective diskriminatif menjaga informasi untuk membedakan target. Autoencoder linear $\approx$ PCA.

Latent space bukan selalu “semantik manusia”

Koordinat laten adalah hasil optimasi terhadap objective tertentu.



$z \rightarrow R z$ (R ortogonal): $\|Rz_i - Rz_j\| = \|z_i - z_j\|$, $\cos$ tidak berubah

Sumbu individual

sering tidak punya arti yang mudah dinamai; koordinat bisa dirotasi tanpa mengubah perilaku

Direction / subspace

makna lebih sering berada pada arah, neighborhood, atau subruang

Validasi

uji interpretasi secara sistematis (probing, intervensi), bukan dari beberapa contoh

Jangan menamai dimensi laten hanya karena korelasi kecil pada beberapa contoh.

Bagaimana mengevaluasi *representation*?

Tidak ada satu metrik universal. Evaluasi harus sesuai tujuan.

Reconstruction

MSE / PSNR / SSIM bila tujuan kompresi atau generatif.

Explained variance

Untuk PCA / kompresi linear.

Retrieval

Recall@K, Precision@K, mAP, nDCG.

Clustering

Silhouette, ARI, NMI bila ada ground truth relevan.

Downstream task

Accuracy/F1/AUC/error setelah linear probe atau fine-tuning.

Robustness

Noise, domain shift, perturbasi, stabilitas subgroup.

Representation “bagus” harus didefinisikan relatif terhadap fungsi yang ingin dipertahankan.

Protokol evaluasi: retrieval dan linear probe

Definisi metrik yang sering dipakai, dengan contoh hitung sederhana.

Recall@K #relevan di top-K / #relevan total

0.67

Recall@5

Precision@K #relevan di top-K / K

0.40

Precision@5

nDCG@K DCG@K / IDCG@K, $DCG = \sum rel_i / \log_2(i + 1)$

0.70

nDCG@5

rank	1	2	3	4	5
relevan?	✓	X	✓	X	X

Contoh: total relevan = 3 (satu di rank 7).

LINEAR PROBE

- Bekukan encoder, ekstrak embedding
- Latih pengklasifikasi linear di atasnya
- Laporkan performa pada data held-out
- Bandingkan dengan baseline (PCA, TF-IDF, acak)

Jangan memilih encoder atau k dengan test set.

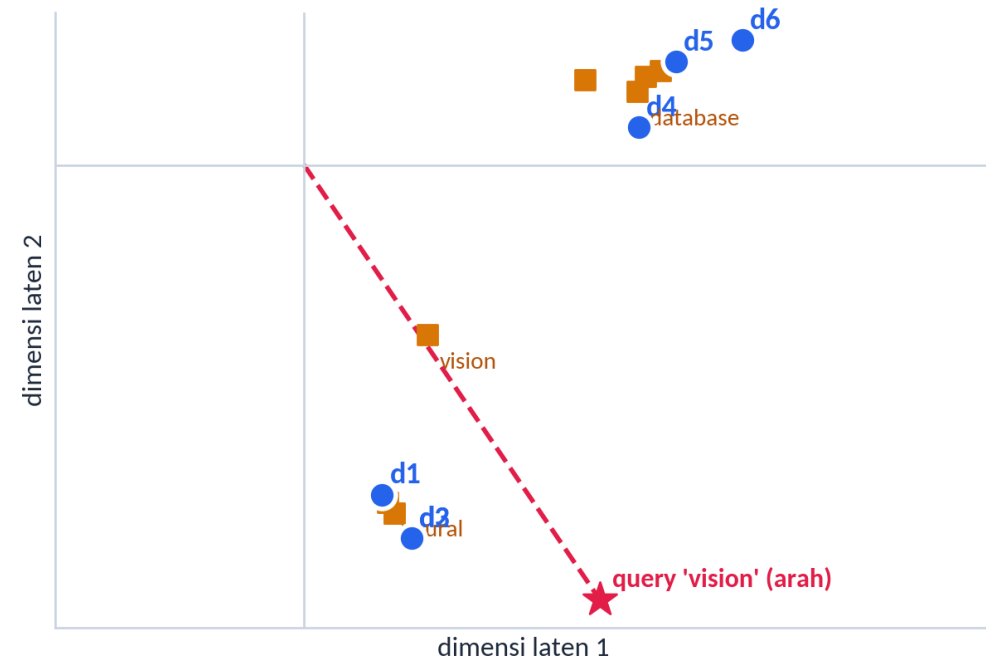
LSA dalam angka: mengapa ruang laten membantu retrieval

Korpus mini: 8 term × 6 dokumen. Query hanya berisi kata “vision”.

term	d1	d2	d3	d4	d5	d6
neural	2	1	1	0	0	0
network	1	2	1	0	0	0
image	1	1	2	0	0	0
vision	0	1	1	1	0	0
database	0	0	0	2	1	1
query	0	0	0	1	2	1
index	0	0	0	1	1	2
transaction	0	0	0	0	1	2

query “vision”	d1	d2	d3	d4	d5	d6
cosine mentah	0.00	0.38	0.38	0.38	0.00	0.00
cosine laten (k = 2)	0.93	0.95	0.95	0.47	0.32	0.32

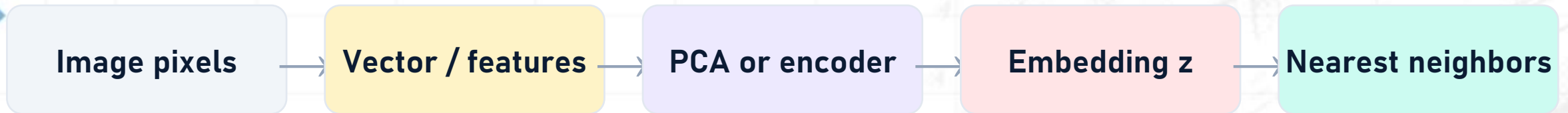
$$\text{dokumen} = V_k \Sigma_k, \text{ term} = U_k \Sigma_k$$



d1 tidak memuat “vision” (mentah 0.00), tetapi ditemukan di ruang laten (0.93): sinonim/ko-okurensi tertangkap.

Case study terpadu: citra

Dari piksel menuju representasi ringkas dan retrieval.



Jika memakai PCA

- Linear compression
- Cepat, deterministic
- Menjaga variance global

Jika memakai deep encoder

- Nonlinear features
- Objective dapat task-specific
- Membutuhkan training/data

Insight

Metode klasik tetap penting sebagai baseline: jika model kompleks tidak mengalahkan PCA secara berarti, kompleksitasnya perlu dipertanyakan.

Terima Kasih

Yuita Arum Sari, S.Kom., M.Kom., Ph.D.
yuita@ub.ac.id

Versi Dokumen:

Tanggal: 15; Bulan: September; Tahun: 2026

© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id